

Ontwerp van een geautomatiseerd systeem voor security monitoring en incident response in SaaS omgevingen binnen de DevOps-infrastructuur van Devinity.

Sem Demoen.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. S. Samyn

Co-promotor: Dhr. T. Neels

Instelling: Devinity BV.

Academiejaar: 2025–2026

Tweede examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Ik koos dit onderwerp vanuit een interesse in security die al langer aanwezig was, maar die tijdens mijn stage bij Devinity een concrete invulling kreeg. Wat me opviel toen ik de werking van het bedrijf beter leerde kennen, was niet zozeer een gebrek aan tools, maar een gebrek aan samenhang tussen die tools. Logs bleven ongelezen, meldingen kwamen binnen zonder prioriteit, en er was geen gestructureerde manier om op incidenten te reageren. Dat is het soort probleem waarbij je als security specialist tegelijk denkt: dit is op te lossen, en: waarom is dit er nog niet? Die combinatie van herkenbaarheid en haalbaarheid maakte het een logische keuze als onderwerp voor mijn bachelorproef.

Ik wil in de eerste plaats mijn promotor bedanken voor de begeleiding doorheen het proces, de feedback op de tussentijdse versies en de vrijheid om zelf keuzes te maken over de richting van het onderzoek. Daarnaast gaat mijn dank uit naar de collega's bij Devinity die tijd vrijmaakten voor gesprekken, vragen beantwoorden over de bestaande toolset en openstonden voor het uitproberen van nieuwe dingen op de QA-omgeving. Zonder die toegang en dat vertrouwen was het proof-of-concept niet mogelijk geweest.

Tot slot wil ik mijn omgeving bedanken voor de steun en het geduld tijdens de drukke momenten van dit semester.

Samenvatting

De toenemende adoptie van SaaS-platformen en DevOps-praktijken brengt heel wat uitdagingen mee voor security monitoring. Organisaties werken met gedecentraliseerde infrastructuren waarin logs, alerts en events verspreid zitten over meerdere tools zonder onderlinge koppeling. Voor Devinity, een SaaS-gericht bedrijf, vertaalt zich dat in een situatie waarbij security-events niet systematisch worden gedetecteerd, incident response voornamelijk reactief verloopt en meetpunten om de effectiviteit van de aanpak te beoordelen volledig ontbreken.

De centrale onderzoeksvraag van deze bachelorproef luidt: hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen de SaaS-omgeving van Devinity effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur? Het doel is een werkend proof-of-concept te bouwen dat de haalbaarheid van zo'n aanpak aantoonst binnen de bestaande toolstack van het bedrijf, zonder de infrastructuur te vervangen.

Het onderzoek verloopt in vier fasen. In de probleemanalyse worden de concrete knelpunten in kaart gebracht via observaties en gesprekken met het team. Op basis daarvan wordt een gelaagde architectuur ontworpen. Make werkt als ingestie- en filterlaag voor twee afzonderlijke scenario's: één voor de verwerking van Azure WAF-security-events en één voor Azure Service Health alerts, waarbij dat laatste scenario op expliciete vraag van de developers werd opgenomen en rechtstreeks naar incident response wordt doorgestuurd zonder tussenkomst van het centrale platform. Azure Log Analytics als centraal opslag- en analyseplatform, Azure Monitor Workbooks als visualisatielaag en monday.com in combinatie met Outlook voor verbeterde incident response. Vervolgens wordt deze architectuur uitgewerkt tot een werkend proof-of-concept dat de Azure WAF als primaire databron gebruikt, via Azure Monitor Alert Rules, KQL-queries, een Data Collection Rule en een Make-scenario met routeringslogica op basis van de ernst van een event. In de vierde fase worden de resultaten geëvalueerd aan de hand van drie KPI's.

De gemiddelde detectietijd bedraagt 187 seconden, consistent binnen het 5-minutenvenster van de Alert Rules. De gemiddelde totale responstijd van trigger tot notificatie of workitem is 23,9 seconden, waarbij Make-verwerking gemiddeld slechts 2,8 seconden inneemt. Over vijf testrondes werd een filterratio van 98,5% gemeten: van 3 857 ruwe events bereikten er slechts 59 de centrale opslag, resulteerden er 8 in een workitem en 2 in een e-mailnotificatie. Die hoge ratio is te verklaren door het beperkte

verkeersvolume in de QA-omgeving, maar de filterlogica functioneert zoals ontworpen.

De resultaten tonen aan dat een effectievere aanpak van security monitoring haalbaar is zonder nieuwe tools aan te schaffen of de bestaande infrastructuur te vervangen. De meerwaarde voor het werkveld zit in het feit dat de architectuur generiek is opgezet en overdraagbaar naar andere SaaS- en DevOps-omgevingen die met gelijkaardige uitdagingen kampen. De voornaamste beperking is dat de validatie beperkt bleef tot de QA-omgeving met gesimuleerde testdata. Cross-source correlatie en productievalidatie blijven aanbevelingen voor een volgende fase.

Inhoudsopgave

Lijst van figuren	xi
Lijst van tabellen	xii
Lijst van codefragmenten	xiii
1 Inleiding	1
1.1 Probleemstelling	2
1.2 Onderzoeksvraag	3
1.3 Onderzoeksdoelstelling	4
1.4 Opzet van deze bachelorproef	5
2 Stand van zaken	6
2.1 Context en Probleemstelling	6
2.1.1 Uitdagingen bij moderne netwerkarchitecturen	6
2.1.2 Preventie versus detectie	7
2.1.3 Architectuur van Intrusion Detection en Prevention Systems	7
2.2 Fundamenten van Security Monitoring en Logbeheer	8
2.2.1 Definitie en doel van security monitoring	8
2.2.2 Security monitoring in het NIST Cybersecurity Framework	9
2.2.3 Logbeheer en logmanagementprocessen	9
2.3 Detectiemechanismen en Analyse	10
2.3.1 Intrusion Detection Systems (IDS) en Intrusion Prevention Systems (IPS)	10
2.3.2 Signature-based detectie	11

2.3.3	Anomaly-based detectie	11
2.3.4	Indicators and precursors	11
2.3.5	Beperkingen van detectiesystemen	12
2.4	SIEM-systemen en architectuurprincipes	13
2.4.1	Definitie en kerncomponenten	13
2.4.2	Visualisatie en rapportage voor stakeholders	14
2.4.3	Complexiteit, schaalbaarheid en configuratie-uitdagingen	14
2.5	Incidentmanagement binnen Security Monitoring.	15
2.5.1	Security incident definitie	15
2.5.2	Incident response lifecycle.	15
2.5.3	Incident response volgens het SANS-model	16
2.5.4	SOAR en Geautomatiseerde Incident Response	17
2.6	Brug naar het eigen onderzoek.	17
3	Methodologie	19
3.1	Fase 1: Probleemanalyse	19
3.2	Fase 2: Architectuurontwerp.	20
3.3	Fase 3: Proof-of-concept ontwikkeling	21
3.4	Fase 4: Evaluatie.	21
4	Fase 1: Probleemanalyse	23
4.1	Inleiding	23
4.2	Huidige toolset en werkwijze	23
4.2.1	New Relic	24
4.2.2	Make	24
4.2.3	Monday.com	25
4.2.4	Azure Portal Tools	25
4.2.5	Microsoft Notifications	26

4.2.6	Analyse van logging en monitoring.	26
4.2.7	Firewallconfiguratie en securityregels	27
4.2.8	Verspreiding van tools en gebrek aan centralisatie.	28
4.2.9	Huidige aanpak van incident response	29
4.3	Probleemstelling	30
4.3.1	Conclusie	31
5	Fase 2: Architectuurontwerp	33
5.1	Inleiding	33
5.2	Architectuuroverzicht	33
5.3	Laag 1: Databronnen	34
5.4	Laag 2: Data-ingestie, filtering en verwerking	35
5.4.1	Aansluiting bij de bestaande werking.. . . .	36
5.4.2	Vergelijking met alternatieven.. . . .	36
5.4.3	Toekomstgerichtheid en uitbreidbaarheid.. . . .	37
5.4.4	Initiële filtering en selectie.	37
5.5	Laag 3: Centraal platform voor verwerking en analyse.	38
5.5.1	Aansluiting bij de bestaande infrastructuur.	38
5.5.2	Kostenbeheer en licenties.. . . .	39
5.5.3	Veiligheid en data governance.. . . .	39
5.5.4	Schaalbaarheid en toekomstbestendigheid.. . . .	39
5.5.5	Analyse en correlatie	39
5.5.6	Visualisatie en rapportering	40
5.6	Incident response.	40
5.7	Evaluatie ten opzichte van de doelstellingen	40
5.8	Visuele voorstelling van de architectuur	42
5.9	Conclusie	43

6	Fase 3: Proof-of-concept ontwikkeling	44
6.1	Inleiding	44
6.2	Testdata en simulatiescenario's	44
6.3	Laag 1: Databron	45
6.4	Laag 2: Data-ingestie, filtering en verwerking via Make	46
6.4.1	Koppeling tussen Azure Monitor en Make	46
6.4.2	Make-scenario: opbouw en werking	47
6.4.3	Initiële filtering via de eerste Router-module	49
6.4.4	Verwerking per pad	49
6.4.5	Severitybepaling	50
6.4.6	Make-scenario: Azure Service Health alerts	51
6.5	Laag 3: Centrale verwerking in Azure Log Analytics	53
6.5.1	Opslag en normalisatie via Data Collection Rule	53
6.6	Visualisatie via Azure Monitor Workbooks	54
6.7	Incident response via Make, Monday.com en Outlook	55
6.8	Mogelijke uitbreidingen	56
6.8.1	Anomaliedetectie op basis van historisch gedrag	56
6.8.2	Geautomatiseerde blokkering van verdachte IP-adressen	57
6.8.3	Periodieke rapportering	57
6.9	Conclusie	57
7	Fase 4: Evaluatie	59
7.1	Inleiding	59
7.2	Evaluatie per KPI	59
7.2.1	KPI 1: Detectiesnelheid	59
7.2.2	KPI 2: Responstijd	61
7.2.3	KPI 3: Verhouding relevante versus irrelevante meldingen	63
7.3	Koppeling aan de deelvragen	65

7.4	Aanbevelingen voor optimalisatie	66
7.5	Conclusie	67
8	Conclusie	69
A	Onderzoeksvoorstel	73
A.1	Inleiding	74
A.2	Literatuurstudie	76
A.3	Methodologie	78
A.4	Verwacht resultaat, conclusie	80
	Bibliografie	81

Lijst van figuren

2.1	Architectuur van een IDPS	8
2.2	SIEM kerncomponenten	14
2.3	Incident response life cycle CSF 2.0	16
3.1	Overzicht van de methodologie in vier fasen.	22
4.1	Overzicht van de methodologie in vier fasen.	31
6.2	Overzicht van het Make-scenario voor de verwerking van security-events	48
6.3	Filterconfiguratie van de eerste Router-module: opsplitsing tussen WAF- firewalllogs en foutmeldingen op basis van het veld <code>data.essentials.alertRule</code>	49
6.5	Overzicht van het Make-scenario voor de verwerking van Azure Service Health alerts	52
6.6	Operationeel dashboard in Azure Monitor Workbooks op basis van de gefilterde <code>WAFFilteredEvents_CL</code> -tabel	54
6.7	Operationeel dashboard in Azure Monitor Workbooks op basis van de gefilterde <code>WAFFilteredEvents_CL</code> -tabel	54
6.8	Operationeel dashboard in Azure Monitor Workbooks op basis van de gefilterde <code>WAFFilteredEvents_CL</code> -tabel	55
6.9	Operationeel dashboard in Azure Monitor Workbooks op basis van de gefilterde <code>WAFFilteredEvents_CL</code> -tabel	55
6.10	Automatisch aangemaakt workitem in Monday.com na een WAF-alert	56
7.1	Detectietijd per meting ten opzichte van de 5-minutengrens (300,s) en het gemiddelde (187,s)	61
7.2	Opgesplitste responstijden per stap voor tien metingen (gestapeld staaf- diagram)	63

7.3 Trechterwerking van de filteringstrategie per testronde: van ruwe events naar gerichte acties	64
---	----

Lijst van tabellen

2.1	Uitdagingen van IDS-systemen	13
6.1	Drempelwaarden voor severitybepaling op basis van OWASP CRS anomaly score	51
6.2	Drempelwaarden voor severitybepaling op basis van herhaalde foutieve responses	51
7.1	Meetresultaten detectiesnelheid per testscenario (n = 10)	60
7.2	Opgesplitste responstijden per stap in de Make-pipeline (n = 10)	62
7.3	Filterefficiëntie over vijf testrondes op de QA-omgeving	64

Lijst van codefragmenten

6.1	KQL-query voor de Azure Monitor Alert Rule op geblokkeerde WAF-events	46
6.2	KQL-query voor de Security-Error-Alert Rule	47

1

Inleiding

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur, en bouwt hier verder op aan de hand van een uitgewerkt proof-of-concept. Het onderzoek vertrekt van een casus bij Devinity, een SaaS-gericht bedrijf dat momenteel meerdere tools/platformen gebruikt voor security monitoring en incidentdetectie. Dit leidt tot problemen zoals vertraagde detectie, onvolledige rapporten en inconsistentie in het afhandelen van incidenten.

De doelgroep bestaat uit het DevOps- en securityteam binnen Devinity, die baat hebben bij een gecentraliseerd en geautomatiseerd systeem dat de operationele efficiëntie verhoogt en risico's beter beheersbaar maakt. Binnen de scope van dit onderzoek wordt gefocust op de Azure Web Application Firewall als primaire data-bron, om de haalbaarheid en diepgang van het proof-of-concept te verzekeren. De architectuur kan zo worden opgezet dat andere bronnen in een latere fase op een gelijkaardige manier kunnen worden geïntegreerd.

Hoewel het onderzoek zich inhoudelijk situeert binnen het domein van Security Information and Event Management (SIEM), wordt geen volledige enterprise-SIEM-oplossing ontwikkeld. In plaats daarvan worden kernprincipes van SIEM toegepast binnen een afgebakende SaaS- en DevOps-context. Concreet ligt de focus op logaggregatie, normalisatie, eventcorrelatie en rapportering.

Deze focus sluit aan bij bestaande literatuur waarin wordt benadrukt dat beveiliging in DevOps-omgevingen geïntegreerd moet worden in zowel de ontwikkel- als deploymentprocessen, zodat kwetsbaarheden sneller kunnen worden gedetecteerd en incidenten efficiënter kunnen worden afgehandeld. Het integreren van

beveiligingscontroles en monitoring binnen de volledige software-levenscyclus wordt daarom beschouwd als een essentieel onderdeel van moderne DevSecOps-praktijken. Het ontwikkelen van een geautomatiseerde en geïntegreerde monitoringoplossing sluit dan ook rechtstreeks aan bij deze aanbevelingen (Souppaya e.a., 2022).

1.1. Probleemstelling

De toenemende adoptie van Software-as-a-Service (SaaS) en DevOps-praktijken heeft geleid tot sterk geautomatiseerde en dynamische omgevingen waarin beveiligingsmonitoring een cruciale rol speelt. Binnen zulke omgevingen worden dan ook continu grote hoeveelheden securitydata gegenereerd die op een efficiënte en gestructureerde manier verwerkt moeten worden, wat niet altijd zo simpel blijkt voor bedrijven. Recente rapporten tonen aan dat onder andere data-beveiliging een belangrijke uitdaging vormt voor organisaties (Tuyishime e.a., 2023).

Volgens (Jakku, 2025) vormen monitoring en logging de basis voor effectieve DevOps-praktijken. Monitoring biedt inzicht in de gezondheid van de systemen, prestatie-metingen en potentiële knelpunten, terwijl logging gedetailleerde gegevens vastlegt over gebeurtenissen, fouten en activiteiten.

Devinity vormt hierin geen uitzondering en krijgt dan ook dagelijks te maken met een concrete uitdaging binnen het vlak van security monitoring en incident response in de eigen SaaS- en DevOps-omgeving. Momenteel worden veel logs, events en alerts bekeken op verschillende afzonderlijke tools en platformen. Hierdoor vergroot de kans dat een incident te laat wordt gedetecteerd, rapportages inconsistent of onvolledig zijn, en dat analyses manueel moeten uitgevoerd worden.

Voor de DevOps engineers en functional testers binnen Devinity betekent dit dat ze geen praktisch en gecentraliseerd overzicht hebben over de beveiligingsstatus van de infrastructuur. Daarnaast krijgt het managementteam geen duidelijke en uniforme rapportages, waardoor risicomanagement heel wat minder doelmatig verloopt.

Het ontbreken van een geautomatiseerde en gecentraliseerde oplossing heeft een directe impact op de analytische efficiëntie, de snelheid van incidentdetectie en de kwaliteit van de besluitvorming binnen Devinity.

Dit toont aan dat er binnen Devinity nood is aan een oplossing die logs, events en alerts uit de gebruikte platformen en omgevingen centraal verzamelt, deze analyseert en automatisch rapporteert. Op deze manier kan het bedrijf incidenten sneller detecteren, onnodige waarschuwingen beperken en een duidelijker overzicht krijgen van de beveiligingsstatus binnen het bedrijf.

1.2. Onderzoeksvraag

De onderzoeksvraag luidt als volgt: Hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen de SaaS-omgeving van Devinity effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur?

De **probleemstelling** wordt verder uitgewerkt aan de hand van volgende deelvragen:

1. Welke uitdagingen ervaren SaaS-bedrijven zoals Devinity bij het gebruik van meerdere tools voor security monitoring en incidentdetectie?
2. Hoe worden incidenten momenteel gedetecteerd en gerapporteerd, en welke tekortkomingen bestaan er in snelheid, correlatie en consistentie van rapporten?
3. Welke securitydata (logs, events, alerts) worden beschikbaar gesteld door bestaande systemen en hoe worden deze momenteel gecentraliseerd en geanalyseerd?
4. Welke metrics en KPIs worden nu gebruikt om de effectiviteit van security monitoring en incident response te meten, en welke zijn onvoldoende of inconsistent?
5. Hoe beïnvloedt de huidige aanpak van monitoring en rapportering de operationele efficiëntie en het risicomanagement van SaaSbedrijven?

De uitwerking van de **oplossing** gebeurt aan de hand van onderstaande deelvragen:

1. Welke architectuur en technologieën zijn het meest geschikt om een geautomatiseerd securitymonitorsysteem te ontwikkelen dat past binnen een DevOps-omgeving,
2. Hoe kan eenvoudige anomaly-detectie op login-auditlogs, als aanvulling op rule-based detectieregels, bijdragen aan het identificeren van ongebruikelijke authenticatiepogingen?
3. Welke data-integratieprocessen (APIs, agents, pipelines) zijn het meest efficiënt om logs en securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline te verzamelen en te normaliseren?
4. Hoe kan automatische rapportgeneratie worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten?

5. Welke prestatie-indicatoren (KPIs) kunnen gebruikt worden om het succes van het systeem te meten, zoals detectiesnelheid, foutpositieven of responstijd?
6. Hoe kan de beveiliging en betrouwbaarheid van het geautomatiseerde systeem zelf worden gegarandeerd binnen de bestaande DevOps-pipeline?

1.3. Onderzoeksdoelstelling

Het doel van deze bachelorproef is het ontwikkelen van een proof-of-concept met het vermogen om securitydata uit een geselecteerd SaaS-platform en DevOps-omgeving automatisch te verzamelen en verwerken. Om ervoor te zorgen dat alle relevante gegevens centraal binnen het systeem terechtkomen kunnen verschillende technieken gebruikt worden, waaronder APIs, webhooks en logforwarders. Na het verzamelen van de data is het de bedoeling dat deze testimplementatie de gegevens structureert en normaliseert zodat ze op een efficiënte manier geanalyseerd kunnen worden. Naast de Azure WAF als primaire databron wordt ook Azure Service Health als operationele bron opgenomen, specifiek gericht op de tijdige opvolging van service-incidenten en onderhoudsmeldingen binnen de Azure-infrastructuur, en dit op vraag van de developers binnen Devinity.

Na het verwerken van deze gegevens zal er een automatisch rapport opgesteld worden dat de belanghebbenden uit het bedrijf voorziet van bruikbare informatie over de beveiligingsstatus om zo incidenten te voorkomen. Deze informatie kan eventueel ook KPIs bevatten om de effectiviteit van monitoring en incident response te meten.

Het systeem moet het dan ook mogelijk maken om sneller inzicht te krijgen in mogelijke incidenten, trends te herkennen in security-events en het overzicht van de infrastructuur te verbeteren, zodat het bedrijf geïnformeerde beslissingen kan nemen en de incidentrespons kan optimaliseren.

Concreet is het eindresultaat een werkend prototype (proof-of-concept) dat de voordelen van centralisatie, snelle detectie en consistente rapportage aantoont voor SaaS- en DevOps gerichte bedrijven zoals Devinity.

Om dit doel te realiseren, wordt het onderzoek uitgevoerd in vier fasen, verder beschreven in Hoofdstuk 3:

1. Probleemanalyse
2. Architectuurontwerp
3. Proof-of-concept ontwikkeling

4. Evaluatie

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4 wordt de eerste fase van het onderzoek behandeld: de probleem-analyse. Hierin wordt de huidige situatie onderzocht, bestaande monitoringtools en processen in kaart gebracht.

In Hoofdstuk 5 wordt het architectuurontwerp gepresenteerd, gebaseerd op de bevindingen uit de probleemanalyse. Er wordt ingegaan op de geselecteerde technologieën, de datastromen en de kerncomponenten van het centrale security-systeem.

In Hoofdstuk 6 wordt de implementatie van het proof-of-concept beschreven. Dit hoofdstuk behandelt de opzet van het prototype, de ontwikkelde kernfuncties zoals logverzameling en detectiemechanismen, en de uitgevoerde tests met simulatie- en testdata.

In Hoofdstuk 7 worden de resultaten van het proof-of-concept geanalyseerd. De prestaties en effectiviteit van het systeem worden beoordeeld en er worden conclusies getrokken en aanbevelingen gedaan voor optimalisatie en toekomstig gebruik.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

2.1. Context en Probleemstelling

2.1.1. Uitdagingen bij moderne netwerkachitecturen

De laatste jaren is er een enorme groei in de nood voor complexe netwerkinfrastructuren en cloudgebaseerde diensten binnen bedrijven in de IT-sector. Dit heeft geleid tot een aanzienlijke toename in het aantal beveiligingstools en monitoring-oplossingen (Persistence Market Research, [2025](#)). Ondanks deze evolutie blijkt het voor heel wat organisaties nog steeds moeilijk om een doordachte en doeltreffende architectuur voor security monitoring te ontwerpen en te implementeren (Obregon, [2015](#)). De gevolgen van deze eerder moeizame implementatie uiteten zich onder meer in dataverlies door onvoldoende zichtbaarheid op security-events en netwerkverkeer, verhoogde kost in nieuwe technologieën en nood aan extra personeel om het overweldigend aantal alerts te verwerken (Obregon, [2015](#)).

Het decentraliseren van de virtuele architectuur binnen een onderneming biedt voordelen, zoals verhoogde schaalbaarheid, flexibiliteit en wendbaarheid. Netwerksegmentatie speelt dan ook zeker een belangrijke rol binnen een informatiebeveiligingsstrategie: door het netwerk op te delen in verschillende zones neemt de kans dat een succesvolle aanval zich verspreidt aanzienlijk af, terwijl de zichtbaarheid van het netwerk tegelijkertijd wordt vergroot, *je kan namelijk niet beschermen wat je niet kan zien* (Obregon, [2015](#)). Echter brengt het opsplitsen van een dergelijk netwerk ook een aanzienlijke problemen met zich mee, met name de verhoogde complexiteit bij het beveiligen van de systemen (Billawa e.a., [2022](#)). Het monitoren van elk afzonderlijk systeem binnen een netwerk is doorgaans ook niet haalbaar vanuit een financieel perspectief en kan bovendien leiden tot een verkeerd gevoel

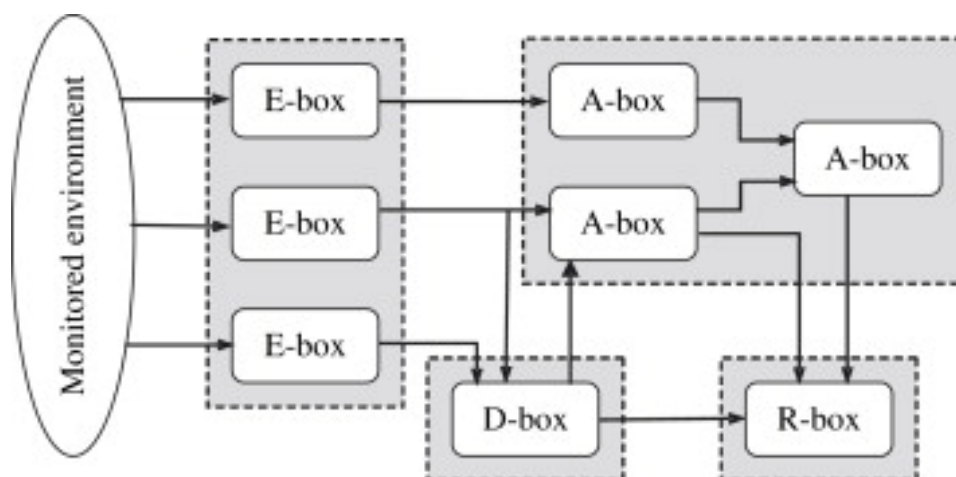
van beveiligingszekerheid binnen de organisatie (Obregon, 2015). Daarom is het van cruciaal belang om op een doordachte manier om te gaan met deze recente fragmentatie.

2.1.2. Preventie versus detectie

"prevention is ideal, but detection is a must" (Dr. Eric Cole, geciteerd in (Obregon, 2015)). Binnen de informatiebeveiliging bestaat er een fundamentele trade-off tussen preventie en detectie bij het implementeren van beveiligingsmaatregelen. Preventie richt zich op het blokkeren van aanvallen vóórdat ze schade veroorzaken, terwijl detectie focust op het identificeren en analyseren van verdachte activiteiten zodra ze plaatsvinden. Volgens de Guide to Intrusion Detection and Prevention Systems (IDPS) kunnen Intrusion Prevention Systems (IPS) automatisch kwaadaardig verkeer blokkeren, bijvoorbeeld door sessies te beëindigen, pakketten te droppen of firewallregels dynamisch aan te passen (Scarfone & Mell, 2007). Intrusion Detection Systems (IDS) daarentegen genereren voornamelijk waarschuwingen, waarna verdere analyse of manuele interventie nodig is. Deze detectiesystemen vermindere het risico op fout-positieven, aangezien ze geen directe impact hebben op het netwerkverkeer. Aan de andere kant is er dan wel meer monitoring en expertise nodig om deze data correct te interpreteren en tijdig op te volgen.

2.1.3. Architectuur van Intrusion Detection en Prevention Systems

IDS-technologie vindt zijn oorsprong bij Denning (Denning, 1987) en werd verder ontwikkeld door Staniford-Chen et al. (Staniford-Chen & Heberlein, 1998). Een belangrijke standaardisering werd later uitgevoerd door CIDF (Common Intrusion Detection Framework) en IDWG (Intrusion Detection Working Group). Zij stelden een basisarchitectuur voor IDPS op, bestaande uit vier functionele modules: E-blocks (Event-boxes), D-blocks (Database-boxes), A-blocks (Analysis-boxes) en R-blocks (Response-boxes) (García-Teodoro e.a., 2009). Een schematische weergave van deze architectuur is te zien in Fig. 2.1.



Figuur 2.1: Schematische weergave van de basisarchitectuur van een IDPS, met de vier functionele modules: E-blocks (Event-boxes), D-blocks (Database-boxes), A-blocks (Analysis-boxes) en R-blocks (Response-boxes) (García-Teodoro e.a., 2009).

De NIST-richtlijnen benadrukken dat een effectieve beveiligingsstrategie niet enkel steunt op preventie of detectie afzonderlijk, maar op een gelaagde benadering waarbij beide complementair worden ingezet. Preventieve mechanismen beperken het aanvalsoppervlak en blokkeren gekende dreigingen, terwijl detectieve systemen zorgen voor zichtbaarheid, analyse van incidenten en het identificeren van nieuwe of geavanceerde aanvallen (Scarfone & Mell, 2007). Naarmate architecturen meer en meer gedecentraliseerd worden, neemt de nood toe aan verhoogde zichtbaarheid en het centraal analyseren van beveiligingsevents, terwijl tegelijkertijd moet worden vermeden dat preventieve controles de processen binnen de organisatie verstoren.

2.2. Fundamenten van Security Monitoring en Logbeheer

2.2.1. Definitie en doel van security monitoring

Vroeger of later zullen de preventiemaatregelen van een netwerk worden overtroffen door aanvallen, op dat moment is het essentieel om in te zetten op detectie en respons. Security monitoring verwijst naar het detecteren van beveiligingsincidenten door het netwerkverkeer te monitoren en analyseren en is één van de meest relevante benaderingen voor netwerkbeveiliging (Fuentes-García e.a., 2021). Het doel van monitoring is om de staat van een gegeven netwerk te controleren om abnormale events te detecteren, en ze daarna tijdig te beheeren (Fuentes-García e.a., 2021).

2.2.2. Security monitoring in het NIST Cybersecurity Framework

Binnen het NIST Cybersecurity Framework situeert security monitoring zich voornamelijk binnen de functies Detect en Respond. De Detect-functie richt zich op het identificeren van afwijkingen en potentiële beveiligingsincidenten. Hierbij wordt onder andere ingezet op het vaststellen van een normale 'baseline' van het systeem- en netwerkgedrag, zodat afwijking sneller gevat kunnen worden. De Respond-functie betrekking heeft op het nemen van gepaste maatregelen om de impact van een incident te beperken en het netwerk zo snel mogelijk te herstellen (recover) (National Institute of Standards and Technology, 2018). Security monitoring vormt dus een belangrijke schakel tussen detectie en respons. Zonder goede monitoring worden incidenten niet tijdig opgemerkt, waardoor een snelle en doordachte reactie moeilijk wordt. Het framework benadrukt daarnaast dat detectie- en responsprocessen regelmatig geëvalueerd en bijgestuurd moeten worden. Op die manier kunnen organisaties zich blijven aanpassen aan nieuwe en steeds veranderende dreigingen en hun algemene weerbaarheid versterken (National Institute of Standards and Technology, 2018).

2.2.3. Logbeheer en logmanagementprocessen

Logmanagement omvat het verzamelen en samenvoegen van loggegevens, het bewaren van originele (ongewijzigde) logs, het analyseren van loginhoud en het presenteren van de resultaten, meestal via zoekfuncties, maar ook in rapportvorm (Chuvakin, 2010). Daarnaast omvat logmanagement ook het ondersteunen van de bijbehorende werkprocessen en het effectief beheren van de loginhoud. Door deze brede aanpak kunnen loggegevens op allerlei manieren worden benut, niet alleen binnen de IT-sector, maar vaak ook daarbuiten, waardoor ze waardevol zijn voor uiteenlopende toepassingen zoals beveiliging en operationeel beheer (Chuvakin, 2010).

Logs kunnen veel verschillende soorten informatie bevatten over de activiteiten en gebeurtenissen binnen systemen en netwerken, die in veel situaties van groot belang kunnen zijn voor de computerveiligheid van organisaties (Kent & Souppaya, 2006). Volgens het National Institute of Standards and Technology is het belangrijk dat organisaties een logmanagementinfrastructuur opzetten en onderhouden, vaak met gecentraliseerde logservers en geschikte opslag van loggegevens (Kent & Souppaya, 2006). Een goed ingerichte infrastructuur maakt het mogelijk loggegevens systematisch te verzamelen, te bewaren en te analyseren, zodat incidenten snel kunnen worden opgespoord en onderzocht, en helpt bij het behouden van overzicht en structuur bij grote hoeveelheden logdata. Hiermee speelt logmanagement een cruciale rol in security monitoring, omdat het organisaties in staat stelt verdachte activiteiten en afwijkingen systemen en netwerken snel te detecteren

en hier tijdig op te reageren.

Het centraliseren en normaliseren van logdata is een belangrijke stap binnen logmanagement, vooral wanneer organisaties te maken hebben met grote hoeveelheden securitydata. Door logs vanuit verschillende systemen en applicaties te centraliseren naar één centrale locatie, zoals een logserver of SIEM-systeem, wordt het overzicht behouden en kan de data effectiever worden geanalyseerd. Normalisatie houdt in dat loggegevens worden omgezet naar een uniform formaat, zodat verschillen in structuur, terminologie en tijdstempels tussen verschillende systemen geen problemen veroorzaken voor analyses (Kent & Souppaya, 2006). Zonder centralisatie en normalisatie kunnen grote volumes aan logdata snel onoverzichtelijk worden, wat leidt tot vertragingen bij het detecteren van afwijkingen en incidenten. Door logs consistent te structureren en op één plek te bewaren, kunnen organisaties sneller verbanden leggen, trends herkennen en verdachte activiteiten detecteren. Daarnaast ondersteunt deze aanpak het gemakkelijker automatiseren van monitoringprocessen, wat cruciaal is voor security monitoring in omgevingen met grote en diverse datasets.

2.3. Detectiemechanismen en Analyse

2.3.1. Intrusion Detection Systems (IDS) en Intrusion Prevention Systems (IPS)

Een Intrusion Detection System (IDS) is software die het detecteren van inbraken automatiseert. Een Intrusion Prevention System (IPS) heeft dezelfde mogelijkheden, maar kan daarnaast ook proberen mogelijke incidenten te stoppen (Scarfone & Mell, 2007). Intrusion Detection and Prevention Systems (IDPS) richten zich vooral op het opsporen van potentiële incidenten. Zo kan een IDPS bijvoorbeeld signaleren wanneer een aanvaller een systeem binnendringt door een bestaande kwetsbaarheid uit te buiten. Daarnaast kan een IDPS incidenten rapporteren, informatie loggen en security-overtredingen herkennen, zodat de organisatie de juiste maatregelen kan nemen (Scarfone & Mell, 2007). Een nadeel van IDPS-technologie is dat deze nooit volledig foutloze detecties kan garanderen. Regelmatig worden fout-positieve meldingen gegenereerd, en het is onmogelijk om zowel foutpositieven als foutnegatieven volledig te elimineren. Het verminderen van het ene type fout leidt namelijk vaak tot een toename van het andere. Het National Institute of Standards and Technology (NIST) geeft volgende info mee:

“Many organizations choose to decrease false negatives at the cost of increasing false positives, which means that more malicious events are detected but more analysis resources are needed to differentiate false positives from true malicious events.” (Scarfone & Mell, 2007).

Dit toont aan dat organisaties voortdurend een evenwicht moeten zoeken tussen detectienauwkeurigheid en de beschikbare middelen voor analyse en opvolging. Door de groeiende afhankelijkheid van informatiesystemen en de toenemende frequentie en impact van aanvallen, zijn IDPS'en tegenwoordig vrijwel onmisbaar geworden in de beveiligingsinfrastructuur van vrijwel elke organisatie, daarom is IDPS een noodzakelijke integratie in een centraal platform voor security monitoring.

2.3.2. Signature-based detectie

Binnen IDPS'en worden er verschillende detectiemethodologieën toegepast, waaronder signature-based detection en anomaly-based detection. Een *signature* is een patroon dat overeenkomt met een gekende aanval. Signatuurgebaseerde detectie heeft als doel om deze patronen te vergelijken met waargenomen gebeurtenissen om mogelijke (gekende) incidenten te identificeren (Scarfone & Mell, 2007). Een voorbeeld hiervan is een inlogpoging met de gebruikersnaam "admin", wat een mogelijke poging tot ongeautoriseerde toegang kan aanduiden. Hoewel signatuurgebaseerde detectie misschien niet de meest geavanceerde strategie is, is ze wel de eenvoudigste om te gebruiken. Het volstaat namelijk om een eenheid van activiteit, zoals een packet of een log entry, te vergelijken met een lijst van signatures via een eenvoudige stringvergelijking (Scarfone & Mell, 2007).

2.3.3. Anomaly-based detectie

Anomaliegebaseerde detectiemechanismen proberen eerst het 'normale' gedrag van het te beschermen systeem in kaart te brengen. Wanneer een waarneming op een bepaald moment te veel afwijkt van dit normale patroon, en een vooraf bepaalde drempel overschrijdt, wordt een alarm geactiveerd (García-Teodoro e.a., 2009). Daarnaast kan men ook specifiek afwijkend gedrag van een systeem vastleggen. In dat geval wordt een alarm geactiveerd wanneer de waargenomen activiteit binnen een bepaalde marge overeenkomt met het vooraf gedefinieerde abnormale gedrag (García-Teodoro e.a., 2009).

2.3.4. Indicators and precursors

Volgens NIST kunnen tekenen van een beveiligingsincident worden onderverdeeld in twee categorieën: *precursors* en *indicators*. Een precursor is een signaal dat aangeeft dat er mogelijk een incident in de toekomst kan plaatsvinden, bijvoorbeeld het scannen van een netwerk op kwetsbaarheden of het gebruik van een bekende exploit. Een indicator daarentegen wijst erop dat een incident al heeft plaatsgevonden of momenteel plaatsvindt, zoals ongewoon netwerkverkeer, meerdere mislukte inlogpogingen of alerts van een IDS/IPS (Nelson e.a., 2025). Het onderscheid

tussen precursors en indicators helpt organisaties om zowel proactief maatregelen te nemen als gepast te reageren op huidige incidenten, waardoor de effectiviteit van een centraal security monitoring platform wordt vergroot.

2.3.5. Beperkingen van detectiesystemen

Hedendaagse detectiesystemen kennen verschillende beperkingen die een invloed kunnen hebben op de effectiviteit van security monitoring. Zoals weergegeven in Tabel 2.1 hebben host-based IDS vaak te maken met beperkte middelen zoals rekenkracht, opslag en batterijcapaciteit. Daarnaast beschikken deze systemen niet altijd over voldoende contextinformatie om verdachte activiteiten correct te beoordelen. Ook kan er vertraging ontstaan wanneer meldingen eerst centraal moeten worden doorgestuurd voordat ze verder verwerkt worden (Raponi e.a., 2019).

Bij network-based IDS komen nog extra uitdagingen kijken. Het detecteren van insider attacks is bijvoorbeeld moeilijker omdat deze aanvallen afkomstig zijn van binnen het netwerk. Daarnaast zorgen versleuteld verkeer en het beperken van false positives en false negatives voor bijkomende complexiteit. Ook factoren zoals de grootte van moderne netwerken, geografische spreiding en dynamische netwerkomgevingen maken het moeilijker om beveiligingsincidenten efficiënt te detecteren en analyseren (Raponi e.a., 2019).

IDS Type	Tier	Uitdagingen
Host-based IDS	Tier 1	Beperkte middelen (rekenkracht, opslag, batterij) Gebrek aan contextinformatie Vertraging bij gecentraliseerde rapportering
Host-based IDS	Tier 2	Gebrek aan contextinformatie Vertraging bij gecentraliseerde rapportering
Network-based IDS	Tiers 1–3	Detectie van insider attacks Samenwerking tussen systemen Verminderen van false positives/negatives Versleuteld netwerkverkeer Detectie van jamming Detectie en mitigatie van DoS-aanvallen
Algemeen	–	Grootschalige netwerken Geografische spreiding Dynamische omgevingen Real-time meldingen Parallel verwerken van alarmen Beheer van false alarms

Tabel 2.1: Overzicht van belangrijke uitdagingen en beperkingen van verschillende IDS-types (Raponi e.a., 2019).

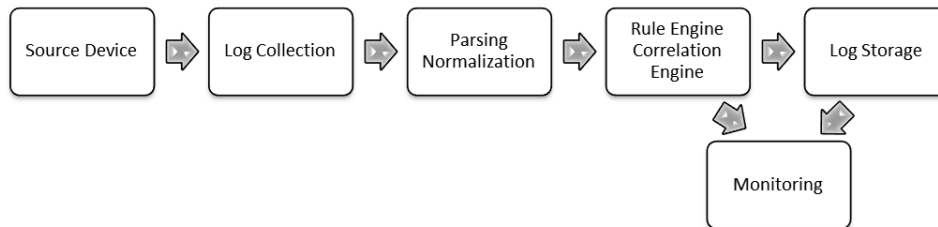
2.4. SIEM-systemen en architectuurprincipes

2.4.1. Definitie en kerncomponenten

Security Information and Event Management (SIEM) systemen spelen een belangrijke rol binnen moderne security monitoring. Een SIEM-platform verzamelt en centraliseert loggegevens en security events uit verschillende bronnen binnen een IT-infrastructuur, waaronder servers en de gebruikte applicaties. Door deze gegevens te verzamelen kan het systeem gebeurtenissen analyseren en verbanden leggen tussen verschillende events om mogelijke beveiligingsincidenten te detecteren (González-Granadillo e.a., 2021).

De kerncomponenten van een SIEM-systeem bestaan uit logverzameling, normalisatie van data, opslag en analyse van gebeurtenissen, zoals weergegeven in Fi-

guur 2.2. Logdata uit verschillende systemen wordt eerst verzameld en vervolgens genormaliseerd zodat deze op een uniforme manier kan worden geanalyseerd. Op basis van deze analyse kunnen correlatieregels worden toegepast om verdachte activiteiten te identificeren en alerts te genereren (González-Granadillo e.a., 2021).



Figuur 2.2: Basiscomponenten van een SIEM-systeem, inclusief logverzameling, normalisatie, opslag, analyse en waarschuwingen. (González-Granadillo e.a., 2021)

2.4.2. Visualisatie en rapportage voor stakeholders

Een belangrijk onderdeel van SIEM-systemen is de rapportering en visualisatie voor stakeholders. Nadat events zijn genormaliseerd en gecorreleerd, kunnen de resultaten worden weergegeven in dashboards en rapporten. Deze tools geven security teams en management een overzicht van de beveiligingsstatus van het netwerk, laten trends zien en helpen bij het prioriteren van incidenten. Door duidelijk inzicht te bieden in actuele bedreigingen en data uit het verleden, kunnen stakeholders beter geïnformeerde beslissingen nemen over mitigatie en strategie (González-Granadillo e.a., 2021). De inzichten uit deze literatuurstudie vormen bovendien een nuttige inspiratie voor de opzet van het proof-of-concept.

2.4.3. Complexiteit, schaalbaarheid en configuratie-uitdagingen

Het implementeren van SIEM-oplossingen brengt echter verschillende uitdagingen met zich mee, vooral voor bedrijven die hun cybersecurity willen versterken. Een belangrijk obstakel is de complexiteit van installatie en operationeel gebruik. SIEM-systemen vereisen een grondig begrip van de netwerkarchitectuur en beveiligingsrichtlijnen om correct te functioneren (Vardalachakis e.a., 2025). Daarnaast is technische expertise nodig om de systemen zo te configureren dat ze data uit verschillende bronnen kunnen verzamelen en analyseren. Het proces wordt nog ingewikkelder wanneer data afkomstig is uit complexe en gevarieerde IT-omgevingen.

Naast de installatie vormt ook de integratie van data een belangrijke uitdaging. Het combineren van gegevens uit verschillende bronnen kan problemen veroorzaken op het gebied van consistentie, organisatie en zelfs de kwaliteit van de data, wat de implementatie sterk vertraagt. Er wordt benadrukt dat een goede voorbereiding en betrokkenheid van stakeholders, samen met het gebruik van adap-

ters of tools voor het versnellen van normalisatie en dataverwerking cruciaal zijn. Door deze obstakels te overwinnen, kunnen organisaties SIEM-systemen optimaal inzetten om hun veiligheid te verbeteren en risico's op cyberaanvallen te verminderen (Vardalachakis e.a., 2025).

2.5. Incidentmanagement binnen Security Monitoring

2.5.1. Security incident definitie

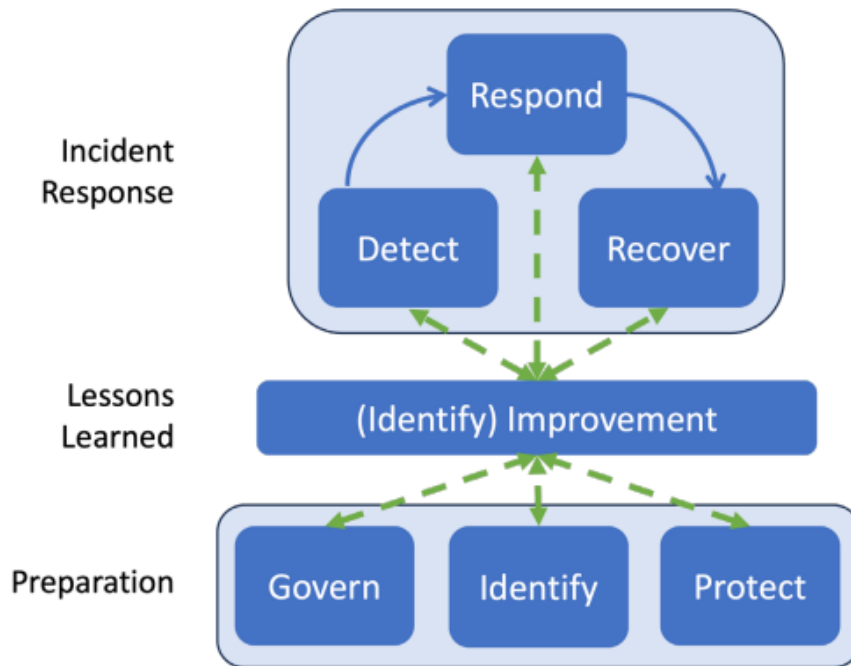
Een cybersecurity event wordt gedefinieerd als een verandering op het gebied van cybersecurity die mogelijk impact kan hebben op de operaties van een organisatie, zoals de missie, capaciteiten of reputatie. Wanneer een dergelijk event daadwerkelijk een significante impact heeft en een organisatie moet reageren om schade te beperken of systemen te herstellen, spreken we van een cybersecurity incident (National Institute of Standards and Technology, 2018). Deze definitie benadrukt het verschil tussen gewone gebeurtenissen en situaties die een daadwerkelijke respons en herstelmaatregelen vereisen, en vormt de basis voor incident detection en management binnen beveiligingssystemen.

2.5.2. Incident response lifecycle

De incident response life cycle kan worden weergegeven op basis van de NIST Cybersecurity Framework (CSF) 2.0 Functions, die cybersecuritydoelstellingen op het hoogste niveau organiseren (National Institute of Standards and Technology (NIST), 2025). Het model onderscheidt zes functies:

1. **Govern (GV):** Strategie, verwachtingen en beleid voor cybersecurityrisicomanagement worden vastgesteld, gecommuniceerd en gemonitord.
2. **Identify (ID):** De huidige cybersecurityrisico's van de organisatie worden begrepen.
3. **Protect (PR):** Beveiligingsmaatregelen worden ingezet om de risico's te beheersen.
4. **Detect (DE):** Mogelijke cyberaanvallen en compromitteringen worden ontdekt en geanalyseerd.
5. **Respond (RS):** Acties worden ondernomen naar aanleiding van een gedetecteerd cybersecurityincident.
6. **Recover (RC):** Systemen en operaties die door een incident zijn getroffen, worden hersteld.

In dit model ondersteunen Govern, Identify en Protect organisaties bij het voorkomen van incidenten, voorbereiden op incidenten en het beperken van de impact, terwijl Detect, Respond en Recover helpen bij het ontdekken, beheren, prioriteren, beheersen, uitroeien en herstellen van incidenten. Bovendien is continue verbetering ingebouwd via de Improvement Category binnen de Identify-functie, waardoor lessen uit alle activiteiten gebruikt kunnen worden om incident response en cybersecurityrisicomanagement voortdurend te optimaliseren (National Institute of Standards and Technology (NIST), 2025).



Figuur 2.3: Hoog-niveau incident response life cycle model gebaseerd op de zes CSF 2.0 Functions: Govern (GV), Identify (ID), Protect (PR), Detect (DE), Respond (RS) en Recover (RC), inclusief het concept van continue verbetering (National Institute of Standards and Technology (NIST), 2025).

2.5.3. Incident response volgens het SANS-model

Het SANS Institute beschrijft in het *SANS Incident Handler's Handbook* een gestructureerde aanpak voor het behandelen van cybersecurity-incidenten binnen organisaties. Het model bestaat uit zes fasen: preparation, identification, containment, eradication, recovery en lessons learned (Kral, 2012). In de praktijk betekent dit bijvoorbeeld dat organisaties eerst monitoring en procedures opzetten om verdachte activiteiten te detecteren, waarna incidenten worden geanalyseerd en indien nodig systemen tijdelijk worden geïsoleerd om verdere schade te beperken. Vervolgens wordt in de eradication-fase de oorzaak van het incident verwijderd, bijvoorbeeld door malware te verwijderen of kwetsbaarheden te patchen, waarna systemen veilig opnieuw in gebruik worden genomen. Tot slot wordt het incident

geëvalueerd zodat processen en beveiligingsmaatregelen in de toekomst kunnen worden verbeterd. Deze gestructureerde aanpak vormt een sterke basis om verder op te bouwen bij het ontwerpen van systemen voor security monitoring en incident response (Kral, 2012).

2.5.4. SOAR en Geautomatiseerde Incident Response

Automatisering speelt een steeds belangrijkere rol binnen incident response, vooral door het gebruik van concepten zoals Security Orchestration, Automation, and Response (SOAR) (González-Granadillo e.a., 2021). In moderne IT-omgevingen, waar grote hoeveelheden loggegevens, alerts en beveiligingsevents worden gegenereerd, wordt het voor security teams steeds moeilijker om alle meldingen manueel te analyseren en te verwerken. Door repetitieve taken zoals alert triage, het verzamelen van loggegevens uit verschillende systemen en het uitvoeren van initiële containmentmaatregelen te automatiseren, kunnen incident response processen aanzienlijk worden versneld. Zo kan een geautomatiseerd systeem bijvoorbeeld automatisch relevante data verzamelen uit monitoringtools, verdachte IP-adressen blokkeren via firewallregels of een gecompromitteerd account tijdelijk uitschakelen. Hierdoor kunnen securityspecialisten zich meer focussen op complexere analyse en besluitvorming (González-Granadillo e.a., 2021).

Daarnaast zorgt automatisering ervoor dat incidenten op een consistente manier worden behandeld, omdat vooraf gedefinieerde playbooks en workflows telkens dezelfde stappen volgen. Dit vermindert de kans op menselijke fouten en maakt het mogelijk om beveiligingsprocessen efficiënter op te schalen wanneer het aantal incidenten toeneemt. In omgevingen zoals cloud- en SaaS-platformen, waar infrastructuur dynamisch en sterk geautomatiseerd is, kan automatisering bovendien helpen om sneller te reageren op beveiligingsincidenten en de impact ervan te beperken (González-Granadillo e.a., 2021).

2.6. Brug naar het eigen onderzoek

Uit de literatuur blijkt dat moderne netwerkarchitecturen, cloudomgevingen en SaaS-platformen heel wat uitdagingen met zich meebrengen voor security monitoring en incident response. Organisaties worden geconfronteerd met complexe en gedecentraliseerde infrastructuren, een overvloed aan alerts en gemengde logbronnen, waardoor traditionele methoden voor detectie en respons vaak onvoldoende blijken (Chuvakin, 2010; Obregon, 2015; Scarfone & Mell, 2007). Zowel het SANS Incident Response Model als de NIST-richtlijnen benadrukken het belang van gestructureerde processen en voortdurende verbetering, terwijl SOAR-achtige automatiseringsconcepten aantonen dat het opschalen van incidentrespons en het

consistent afhandelen van gebeurtenissen in dynamische IT-omgevingen mogelijk is (González-Granadillo e.a., [2021](#); Kral, [2012](#)).

Ondanks deze inzichten bestaan er nog duidelijke gebreken aan informatie en praktische uitdagingen. Er is bijvoorbeeld beperkte literatuur over de integratie van geautomatiseerde incident response binnen DevOps-omgevingen en SaaS-platformen, waar infrastructuur dynamisch is en snelle implementatie van wijzigingen vereist. Daarnaast blijven schaalbaarheid, real-time analyse van grote hoeveelheden logdata en de afstemming van detectie- en preventiemechanismen op operationele processen knelpunten die praktische implementatie moeilijker maken (Raponi e.a., [2019](#); Vardalachakis e.a., [2025](#)).

Het eigen onderzoek bouwt voort op bestaande literatuur door de principes van het SANS-model en SOAR te combineren binnen een concrete en praktische context van SaaS- en DevOps-omgevingen. Het levert niet enkel een theoretische onderbouwing, maar ontwikkelt ook een proof-of-concept dat als referentie kan dienen voor organisaties die hun security monitoring en incident response willen moderniseren en automatiseren. Op deze manier wordt een duidelijke verbinding gelegd tussen academische inzichten en praktische toepassing binnen moderne IT-infrastructuren.

3

Methodologie

Het onderzoek wordt uitgevoerd in vier fasen, die samen een gestructureerd en iteratief plan van aanpak vormen. Elke fase draagt bij aan het ontwikkelen en evalueren van een proof-of-concept voor security monitoring en incident response binnen een SaaS- en DevOps-context. De fasen bouwen bewust op elkaar voort: de probleemanalyse stuurt het architectuurontwerp, het ontwerp vormt de basis voor de implementatie, en de evaluatie toetst het geheel aan de vooropgestelde doelstellingen en deelvragen.

3.1. Fase 1: Probleemanalyse

In de eerste fase wordt in samenwerking met Devinity de huidige situatie grondig onderzocht. De bestaande monitoringtools, werkwijzen en knelpunten worden in kaart gebracht via observaties, gesprekken met collega's en een eerste verkenning van beschikbare logs en systemen. Ook worden bestaande incidenten en de manier waarop deze momenteel worden opgevolgd geanalyseerd. Die analyse is niet louter beschrijvend: het doel is een scherp en onderbouwd beeld te krijgen van waar de grootste pijnpunten zich bevinden, zodat het architectuurontwerp in de volgende fase daar gericht op kan inspelen.

Meer bepaald wordt onderzocht welke uitdagingen Devinity ervaart bij het gebruik van meerdere tools voor security monitoring (deelvraag P1), hoe incidenten momenteel worden gedetecteerd en gerapporteerd en welke tekortkomingen daarin bestaan op vlak van snelheid, correlatie en consistentie (deelvraag P2), welke securitydata beschikbaar is vanuit de bestaande systemen en hoe deze momenteel worden gecentraliseerd (deelvraag P3), welke metrics en KPI's vandaag worden ge-

bruikt en waar deze tekortschieten (deelvraag P4), en hoe de huidige aanpak de operationele efficiëntie en het risicomanagement beïnvloedt (deelvraag P5).

Het resultaat van deze fase is een gedetailleerde probleemstelling die de basis vormt voor het architectuurontwerp. Hierbij wordt ook expliciet rekening gehouden met de niet-disruptieve doelstelling: de analyse brengt niet alleen tekortkomingen in kaart, maar ook welke bestaande tools en werkwijzen behouden en verder benut kunnen worden.

3.2. Fase 2: Architectuurontwerp

Op basis van de probleemanalyse wordt een architectuur ontworpen voor de centrale verzameling en verwerking van securitydata. Verschillende technologieën worden afgewogen en geselecteerd op basis van hun aansluiting bij de bestaande infrastructuur van Devinity, hun schaalbaarheid en de mate waarin ze integreerbaar zijn zonder grote aanpassingen aan de bestaande werking. De datastromen worden gedefinieerd via pipelines zoals API-polling, webhooks en logforwarders, en voor elke keuze wordt expliciet verantwoord waarom die boven alternatieven de voorkeur krijgt. De architectuur bouwt voort op de kernprincipes van SIEM-systemen, namelijk logaggregatie, normalisatie, eventcorrelatie en rapportering, en houdt rekening met de NIST-richtlijnen voor intrusion detection en prevention (Scarfone & Mell, 2007).

Concreet wordt in deze fase bepaald welke architectuur en technologieën het meest geschikt zijn voor een geautomatiseerd securitymonitoringsysteem binnen een DevOps-omgeving (deelvraag O1), welke data-integratieprocessen het meest efficiënt zijn om logs en securitydata te verzamelen en te normaliseren (deelvraag O3), en hoe de beveiliging en betrouwbaarheid van het systeem zelf kunnen worden gegarandeerd binnen de bestaande pipeline (deelvraag O6). Ook wordt in deze fase al nagedacht over de KPI's die later in de evaluatiefase zullen worden gebruikt (deelvraag O5).

Een belangrijke randvoorwaarde in dit ontwerp is dat de oplossing zo weinig mogelijk ingrijpt op de bestaande infrastructuur. Ontwerpkeuzes worden daarom bewust afgestemd op de tools en werkwijzen die al aanwezig zijn binnen Devinity, zoals Make voor automatisatie en Azure voor infrastructuur en beveiliging. Het resultaat van deze fase is een gedocumenteerd systeemontwerp met een beschrijving van de gekozen architectuur, de databronnen, de ingestiepipeline en het centrale analyseplatform.

3.3. Fase 3: Proof-of-concept ontwikkeling

In deze fase wordt het ontwerp omgezet naar een werkend prototype binnen de geselecteerde omgeving. Kernfuncties zoals logverzameling, regelgebaseerde detectie, eenvoudige anomaly-detectie en automatische rapportage worden stap voor stap geïmplementeerd en getest. Die iteratieve aanpak laat toe om tussentijds bij te sturen wanneer bepaalde ontwerpkeuzes in de praktijk anders uitpakken dan verwacht.

Testen worden uitgevoerd met gesimuleerde data en ongevoelige logdata uit testomgevingen, bestaande uit authenticatielogs, auditlogs, API-events en doelbewust gegenereerde security-events zoals mislukte logins, om realistische scenario's te kunnen simuleren zonder productiedata te gebruiken of de live omgeving te verstoren. Zo wordt uitgewerkt hoe rule-based detectie op afwijkende HTTP-statuscodes in de AGWAccessLogs zoals herhaalde 401- en 403-responses, als aanvulling op regelgebaseerde detectie, kan bijdragen aan het identificeren van mogelijk ongeautoriseerde toegangspogingen (deelvraag O2), en hoe automatische rapportgeneratie kan worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten (deelvraag O4). Tegelijk wordt in de praktijk getoetst of de gekozen data-integratieprocessen effectief werken zoals voorzien in het ontwerp (deelvraag O3).

Het resultaat van deze fase is een werkend proof-of-concept dat de centralisatie, analyse en rapportage van securitydata aantoont binnen de omgeving van Devinity.

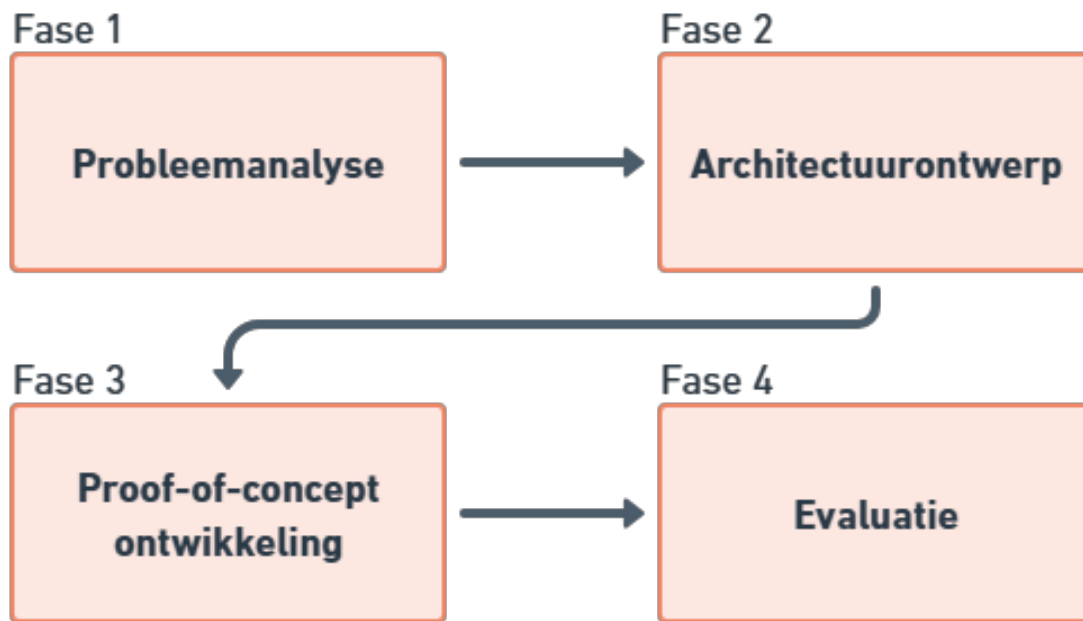
3.4. Fase 4: Evaluatie

In de laatste fase worden de prestaties van het proof-of-concept geëvalueerd aan de hand van vooraf gedefinieerde KPI's, zoals detectiesnelheid, responstijd en de verhouding tussen relevante en irrelevante meldingen. Die KPI's werden niet ad hoc vastgelegd, maar bewust gekozen op basis van de tekortkomingen die in de probleemanalyse naar voren kwamen, zodat de evaluatie een directe vergelijking mogelijk maakt tussen de oude en de nieuwe situatie. De metingen worden uitgevoerd over meerdere testrondes om toevallige uitschieters te minimaliseren en een betrouwbaar beeld te geven van de gemiddelde en maximale prestaties van het systeem.

Deze fase sluit de cirkel met de probleemgerichte deelvragen door na te gaan in welke mate de geïmplementeerde oplossing de eerder vastgestelde tekortkomingen effectief aanpakt. Zo wordt getoetst of incidenten sneller en consistentier worden gedetecteerd (deelvraag P2), of de gecentraliseerde aanpak zorgt voor een

beter overzicht van securitydata (deelvraag P3), en of de gedefinieerde KPI's een objectieve meting van de effectiviteit mogelijk maken (deelvraag P4 en O5). Ook wordt beoordeeld of de oplossing de operationele efficiëntie verbetert ten opzichte van de huidige situatie (deelvraag P5).

Het resultaat van deze fase is een evaluatierapport met conclusies en aanbevelingen voor verdere uitbouw van het systeem binnen de infrastructuur van Devinity.



Figuur 3.1: Schematisch overzicht van de vierfasen-methodologie van dit onderzoek.

De aanpak bouwt voort op de NIST-aanbevelingen voor intrusion detection en prevention systemen (Scarfone & Mell, 2007), waarin het vastleggen en analyseren van security-events centraal staat voor effectieve detectie en respons. Tegelijkertijd baseert de architectuurkeuze zich op kerncomponenten van SIEM-systemen, zoals gecentraliseerde logverzameling, normalisatie en event-monitoring (González-Granadillo e.a., 2021).

4

Fase 1: Probleemanalyse

4.1. Inleiding

In deze fase wordt de huidige situatie binnen Devinity geanalyseerd met betrekking tot security monitoring en incident response in hun SaaS-omgevingen. Het doel van deze analyse is om inzicht te krijgen in de bestaande infrastructuur, gebruikte tools en de belangrijkste knelpunten. Op basis hiervan kan in latere fases een gericht en haalbaar ontwerp worden uitgewerkt.

De informatie in deze probleemanalyse is verzameld via observaties, gesprekken met collega's en een eerste verkenning van beschikbare logs en systemen. Bij het uitwerken van een oplossing staat niet het vervangen van de bestaande omgeving centraal, maar wel het zo weinig mogelijk ingrijpend verbeteren van wat er al is. Ontwerpkeuzes worden daarom bewust afgestemd op de huidige workflow en infrastructuur van Devinity, zodat de oplossing haalbaar blijft en de dagelijkse werking zo min mogelijk wordt verstoord.

4.2. Huidige toolset en werkwijze

Binnen Devinity wordt gebruikgemaakt van verschillende tools voor monitoring, security en incidentopvolging. Voor observability en monitoring wordt onder andere *New Relic* ingezet. Daarnaast wordt *Make* gebruikt om bepaalde processen te automatiseren.

4.2.1. New Relic

New Relic kan het best omschreven worden als een AI-powered 'observability'-platform dat belanghebbenden helpt bij het plannen, implementeren en draaiende houden van software. Eén van de kernfuncties van dit platform is het centraal verzamelen en analyseren van logs en metrics afkomstig uit verschillende onderdelen van de infrastructuur. Hierdoor kunnen developers en analisten real-time inzicht krijgen in het gedrag van de applicaties en kunnen afwijkingen snel worden opgemerkt via alerts en dashboards, waardoor potentiële problemen geïdentificeerd kunnen worden. Binnen het bedrijf wordt er momenteel echter niet optimaal gebruikgemaakt van deze mogelijkheden. Veel functionaliteiten van New Relic blijven onbenut en de verzamelde data wordt momenteel nog niet ingezet voor een proactieve monitoringstrategie.

4.2.2. Make

Make wordt binnen de organisatie ingezet als automatisatietool om verschillende systemen met elkaar te verbinden en specifieke taken binnen het kader van security incident management te vereenvoudigen. Via deze tool kunnen gebeurtenissen, zoals fouten of verdachte activiteiten, automatisch worden doorgezonden vanuit andere applicaties om daarna verder te verwerken. Op basis van vooraf gedefinieerde scenario's kunnen meldingen worden gegenereerd en incidenten worden geregistreerd om volgende stappen op te kunnen starten. Hierdoor ondersteunt Make het sneller signaleren en opvolgen van potentiële incidenten, al gebeurt dit momenteel eerder op basis van losse automatisaties dan binnen een volledig geïntegreerde securitystrategie. Via deze tool worden scenario's opgezet die acties uitvoeren op basis van specifieke triggers. Enkele voorbeelden hiervan binnen het bedrijf zijn:

- Het versturen van notificaties bij herhaalde verwerkingsfouten binnen kritische processen
- Het genereren van meldingen bij fouten in externe integraties en datastromen
- Het detecteren en melden van ontbrekende of inconsistente data
- Het signaleren van synchronisatieproblemen tussen verschillende systemen
- Het automatisch aanmaken van workitems binnen Monday.com voor de opvolging van incidenten

Daarnaast gaven de developers tijdens de probleemanalyse aan dat er ook nood is aan een gestructureerde opvolging van Azure Service Health meldingen. Service-

veranderingen, geplande onderhoudsmomenten en actieve incidenten binnen de Azure-infrastructuur worden momenteel niet systematisch opgevolgd, waardoor het team hier vaak pas achteraf van op de hoogte raakt. Dit vormt een bijkomend aandachtspunt dat in de uitgewerkte oplossing wordt meegenomen.

Hoewel Make op deze manier bijdraagt aan een efficiëntere en deels geautomatiseerde opvolging van processen, biedt het nog geen volledig geïntegreerd kader voor security en monitoring.

4.2.3. Monday.com

Voor incidentopvolging wordt *Monday.com* gebruikt. Monday.com is een cloud-gebaseerd en visueel platform voor werkbeheer dat voornamelijk wordt gebruikt voor het plannen en organiseren van activiteiten en taken. Binnen Devinity wordt het platform, naast de gebruikelijke opvolging van projecten, specifiek ingezet om incidenten te registreren en op te volgen. Wanneer zich een probleem voordoet, worden *via een scenario binnen Make* automatisch workitems aangemaakt, zodat het team op de hoogte wordt gebracht en het incident verder kan worden opgevolgd. Deze aanpak is voornamelijk *reactief*: acties worden pas ondernomen nadat een incident zich heeft voorgedaan en geregistreerd is in het systeem. Hoewel dit zorgt voor een opvolging van incidenten en een overzicht van openstaande workitems, ontbreekt momenteel een proactief onderdeel dat incidenten vroegtijdig signaleert of automatisch corrigerende maatregelen opstart. Hierdoor ligt de nadruk nog sterk op het registreren en volgen van problemen, in plaats van op het voorkomen of snel mitigeren ervan.

4.2.4. Azure Portal Tools

Op vlak van infrastructuur en security wordt gebruikgemaakt van standaard *Azure portal tools*. Een belangrijk onderdeel hiervan is de *Web Application Firewall (WAF)*, waarin zowel managed rules als recent geïmplementeerde custom rules worden toegepast. De managed rules bestaan uit standaardregelsets, zoals *Microsoft Bot-ManagerRuleSet* en *OWASP ruleset*, die samen zorgen voor een brede basisbescherming tegen veelvoorkomende aanvallen, met in totaal een 200-tal regels. De custom rules zijn specifiek ontwikkeld om te voldoen aan de unieke vereisten van de applicaties van Devinity, met nadruk op GraphQL-verkeer en interne API's. Deze regels bestaan uit één of meerdere voorwaarden die, wanneer ze worden vervuld, een specifieke actie activeren. Ze worden allemaal ingesteld als match rules binnen de WAF-policy, zodat de firewall op basis van deze voorwaarden beslissingen kan nemen over het toestaan of blokkeren van verkeer. Voor testdoeleinden in de QA-omgeving kunnen deze custom rules tijdelijk worden uitgeschakeld, zodat de

effecten van de regels kunnen worden geanalyseerd zonder de werking van de productieomgeving te beïnvloeden.

4.2.5. Microsoft Notifications

Naast de WAF wordt voor bredere, globale securitymeldingen ook *Microsoft 365 Defender* gebruikt. Hiermee kunnen e-mailmeldingen worden ingesteld via de Microsoft Defender-portal (Defender XDR email notifications), zodat het team op basis van ingestelde regels berichten ontvangen over nieuwe incidenten of updates van bestaande incidenten. In de praktijk worden deze e-mails echter nauwelijks bekeken, omdat er vaak weinig directe acties mogelijk zijn of omdat de scope van de meldingen te breed is om gericht te kunnen handelen. Hierdoor ontstaan vaak meldingen die wel geregistreerd worden, maar niet verder worden opgevolgd, waardoor mogelijk belangrijke signalen over het hoofd worden gezien. Samen vormen de WAF en Microsoft 365 Defender een combinatie van real-time bescherming en breed overzicht, maar ze bieden nog geen geïntegreerd beeld van de security-status en incidenten binnen het bedrijf.

Logs, meldingen en incidenten zijn verspreid over meerdere systemen, waardoor analyse en prioritering ingewikkelder worden. Er ontbreekt een gecentraliseerd platform waarin correlaties tussen verschillende events gelegd kunnen worden. Hoewel de individuele tools elk hun functie vervullen, is er momenteel geen integratie tussen de systemen, wat resulteert in een gebrek aan een globaal overzicht van alle security-events en monitoringdata. Dit maakt het herkennen van trends, het inschatten van risico's en het proactief reageren op mogelijke dreigingen heel wat moeilijker.

4.2.6. Analyse van logging en monitoring

Eén van de belangrijkste observaties binnen Devinity is dat logging en monitoring nog niet volledig zijn uitgewerkt en grotendeels verspreid plaatsvinden. Hoewel er logs beschikbaar zijn binnen de verschillende systemen (New Relic, de Azure Portal WAF, Microsoft 365 Defender en diverse applicaties), worden deze momenteel niet systematisch verzameld of geanalyseerd. Logs blijven vaak in afzonderlijke systemen staan zoals mailboxen of monday.com workitems, waardoor het moeilijk is om een volledig en samenhangend overzicht te verkrijgen van alle events binnen de infrastructuur. Dit beperkt de mogelijkheden om trends of patronen in incidenten te herkennen, en maakt het snel identificeren van kritieke problemen of verdachte activiteiten een heel stuk ingewikkelder.

Het ontbreken van een centraal platform betekent dat verbanden leggen tussen verschillende events een handmatig en niet-gestandaardiseerd proces is. Analis-

ten moeten informatie uit meerdere bronnen samenbrengen, wat tijdrovend is en foutgevoelig kan zijn. Hierdoor kunnen kleine signalen die duiden op potentiële dreigingen gemakkelijk over het hoofd worden gezien. Bovendien is er geen overzichtelijke prioritering van gebeurtenissen, waardoor het risico bestaat dat belangrijke meldingen niet op tijd worden opgevolgd. Deze verdeelde aanpak maakt het ook lastig om een snelle incident response op te zetten, aangezien relevante context vaak ontbreekt of moeilijk terug te vinden is over de verschillende delen van de architectuur.

Daarnaast ontbreken duidelijke KPI's of meetpunten voor security monitoring. Dit maakt het lastig om objectief te beoordelen of de huidige aanpak effectief is en of de ingezette tools daadwerkelijk bijdragen aan het vroegtijdig detecteren van incidenten. Zonder meetbare indicatoren ontstaan er blinde vlekken in kritieke processen, en wordt het moeilijk om veranderingen te evalueren en de monitoring en veiligheidscontroles continu te verbeteren.

Het feit dat er tot nu toe nog geen (grote) incidenten zijn geweest, zoals aangegeven door de developers, betekent dat de tekortkomingen in logging en monitoring nog niet sterk naar voren zijn gekomen. Hoewel dit op het eerste gezicht geruststellend lijkt, betekent het juist dat er een onzichtbaar risico aanwezig is: *potentiële problemen of kwetsbaarheden kunnen onopgemerkt blijven totdat ze tot een ernstig incident leiden*. In combinatie met de verspreide tools en het gebrek aan centralisatie vormt dit een situatie waarin reactief handelen nog steeds de standaardaanpak blijft, in plaats van een proactieve detectie en preventie van security-problemen. Een structurele verbetering van logging en monitoring, inclusief centralisatie, beschrijven van events en duidelijke meetpunten, is daarom noodzakelijk om een solide en betrouwbaar securitybeheer op te bouwen.

4.2.7. Firewallconfiguratie en securityregels

Binnen Devinity vormt de *configuratie van de Web Application Firewall (WAF)* binnen Azure een cruciaal onderdeel van de security-infrastructuur. Correct ingestelde firewallregels zijn essentieel om zowel de applicaties te beschermen tegen externe aanvallen als de beschikbaarheid en werking van de diensten te waarborgen. Door de verschillen tussen omgevingen, applicaties en type verkeer is het opstellen van consistente en effectieve regels echter complex. Dit maakt firewallbeheer een belangrijk aandachtspunt binnen het huidige securityproces van het bedrijf.

Een concreet probleem, zoals vermeld door het team, is de onzekerheid over welke verzoeken precies toegelaten moeten worden en welke geblokkeerd. Het bepalen van deze grens is een fijne balans: enerzijds moet de firewall streng genoeg zijn om potentiële aanvallen effectief te blokkeren, anderzijds mag de werking en efficiën-

tie van de applicaties niet worden verstoord. Een foutieve configuratie kan zowel securityrisico's als functionele problemen veroorzaken. Deze uitdaging wordt nog moeilijker door de diversiteit aan applicaties en diensten binnen Devinity, elk met hun eigen verkeerspatronen en specifieke eisen.

Een extra uitdagende factor is het GraphQL-verkeer dat naar de verschillende backends gaat. Dit type verkeer vereist vaak meer verfijnde en aangepaste controles, omdat reguliere regels te algemeen zijn om alle mogelijke kwetsbaarheden af te dekken. Daarnaast is het voor het development team vaak onduidelijk welke defaultinstellingen van de firewall actief zijn en hoe deze precies in werking treden. Het gebrek aan inzicht in deze standaardregels vermindert de effectiviteit van het beheer en maakt het gecontroleerd doorvoeren van wijzigingen of nieuwe regels nog ingewikkelder.

Hoewel de meeste securityregels en de basisconfiguratie van de firewall recent op-nieuw zijn opgezet, wordt de verwerking en opvolging van deze informatie nog niet systematisch uitgevoerd. *De huidige workflow is grotendeels gebaseerd op scenario's in Make die incidenten of alerts doorsturen naar workitems in Monday.com.* Dit zorgt wel voor een minimale opvolging, maar er ontbreekt een overzichtelijk en gecentraliseerd proces voor het beheren en evalueren van firewallregels. Hierdoor heeft het team beperkt inzicht in de configuratie, is het moeilijk om wijzigingen bij te houden en kunnen incidenten niet altijd snel of consistent worden afgehandeld.

Er is daarom een duidelijke nood aan een beter gestructureerd proces voor firewall-configuraties en securityregels. Idealiter worden alle regels, alerts en incidenten op een centrale plek verzameld en op een overzichtelijke manier gepresenteerd, zodat belanghebbenden een duidelijk beeld hebben van de configuratie en de status van incidenten. Dit zorgt ervoor dat belangrijke informatie niet over het hoofd wordt gezien, prioriteiten helder zijn en acties efficiënt kunnen worden opgevolgd. Hoewel in dit proof-of-concept niet wordt voorzien in het direct aanpassen van de configuratie, kan het verbeteren van inzicht en overzicht de reactietijd bij incidenten verkorten en de monitoring en opvolging binnen het securityproces van Devinity heel wat verbeteren.

4.2.8. Verspreiding van tools en gebrek aan centralisatie

Een kernprobleem binnen Devinity is de verspreiding van tools binnen de infrastructuur. Monitoring, logging, security en incidentopvolging vinden momenteel plaats in verschillende systemen, waaronder New Relic, de Azure Portal en Monday.com, vaak zonder directe koppeling tussen deze platformen. Elk systeem vervult zijn eigen rol, maar er is geen overkoepelende integratie die alle informatie samenbrengt in één geheel. *Dit gebrek aan centralisatie vormt het belangrijkste*

aandachtspunt binnen deze bachelorproef en zal verder onderzocht worden met als doel een meer geïntegreerde en overzichtelijke aanpak uit te werken.

Door het gebrek aan centralisatie zorg er ook voor dat het meer tijd kost om een volledig beeld te krijgen van een situatie, aangezien gegevens deels manueel moeten worden samengebracht uit verschillende tools. Dit maakt het moeilijker om een duidelijk beeld te vormen van gebeurtenissen en om logs en meldingen correct te interpreteren binnen hun context. Bovendien zijn de belangrijke signalen dan ook niet onmiddellijk zichtbaar binnen het kader van de andere relevante informatie.

Daarnaast maakt het ontbreken van één centraal dashboard of platform zowel de dagelijkse opvolging als analyses ingewikkelder. Het team moeten schakelen tussen verschillende tools om inzicht te krijgen in de status van systemen en incidenten, wat inefficiënt is en kan leiden tot vertragingen in het trekken van conclusies. Ook is er weinig uniformiteit in hoe informatie wordt weergegeven en geïnterpreteerd, wat het moeilijker maakt om snel en consistent te reageren op problemen.

Deze verspreiding heeft ook een impact op de algemene securitystrategie. *Doordat er geen centraal overzicht bestaat, blijft de aanpak opnieuw grotendeels afhankelijk van individuele tools en workflows.* Er is nood aan een oplossing die deze verschillende bronnen samenbrengt en informatie op een gestructureerde en overzichtelijke manier presenteert. Dit zou niet alleen de efficiëntie verhogen, maar ook bijdragen aan een beter inzicht in de volledige securitycontext en een snellere, meer consistente opvolging van incidenten.

4.2.9. Huidige aanpak van incident response

De huidige aanpak van incident response binnen Devinity is voornamelijk reactief. Wanneer zich een probleem voordoet, wordt dit geregistreerd en opgevolgd via tools zoals Monday.com, vaak op basis van meldingen die vanuit andere systemen worden doorgestuurd. Deze werkwijze zorgt ervoor dat incidenten wel opgevolgd worden, maar pas nadat ze zich hebben voorgedaan. Er is momenteel geen geautomatiseerd systeem dat proactief bedreigingen detecteert of automatisch acties onderneemt om incidenten te beperken of te voorkomen.

Daarnaast ontbreekt een duidelijk uitgewerkt plan van aanpak voor incident response. Er zijn geen gestandaardiseerde procedures of workflows die beschrijven hoe met verschillende types incidenten moet worden omgegaan. Hierdoor hangt de aanpak vaak af van de persoon die dat specifieke incident behandelt, wat kan leiden tot inconsistentie en een minder efficiënte opvolging.

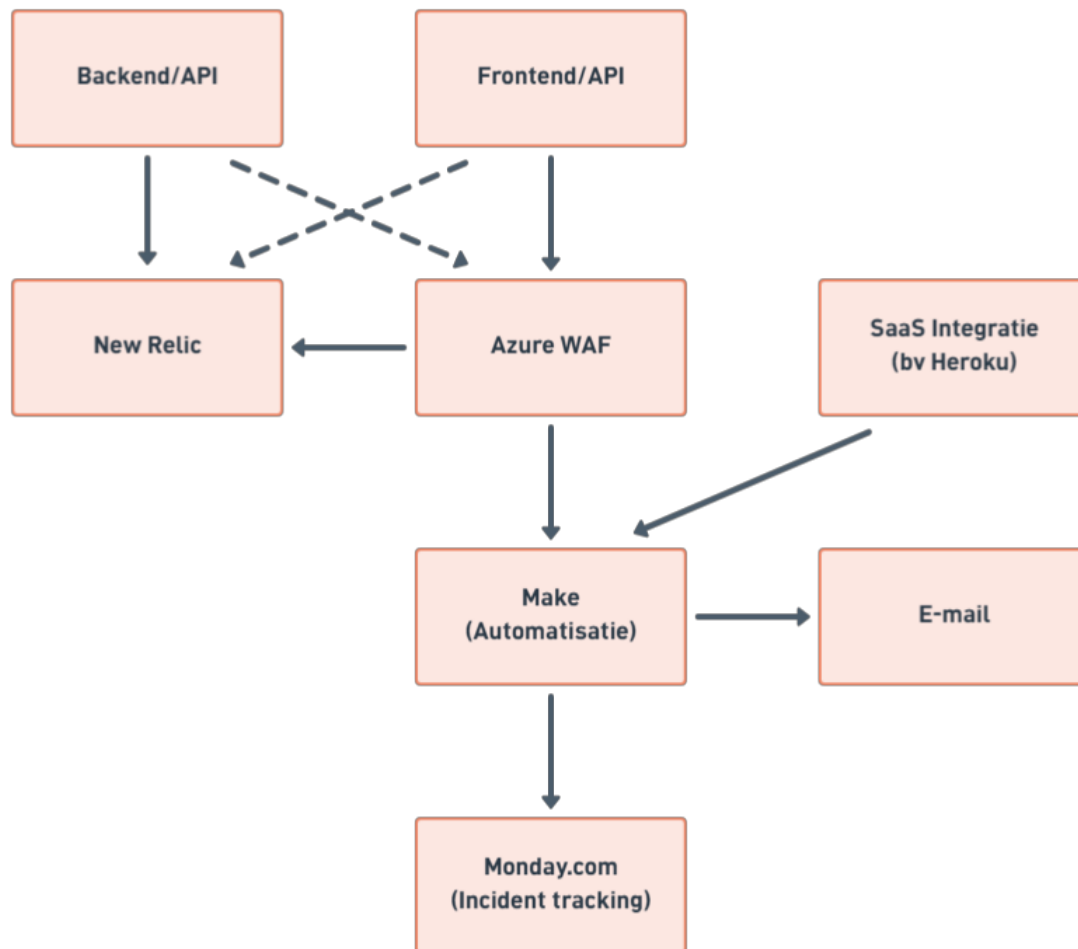
Het ontbreken van een gestructureerde en proactieve aanpak heeft als gevolg dat

incidenten niet altijd op een uniforme en tijdige manier kunnen worden behandeld. Bovendien is het moeilijk om lessen te trekken uit eerdere incidenten, aangezien er geen vast proces is om deze systematisch te analyseren en te documenteren. Ook wordt er momenteel geen overzichtelijke rapportage gegenereerd wanneer een incident of verdachte situatie zich voordoet, waardoor het moeilijk is om achteraf een volledig beeld te krijgen van wat er precies is gebeurd. Dit beperkt de mogelijkheid om de incident response in de toekomst te verbeteren en beter voorbereid te zijn op gelijkaardige situaties.

4.3. Probleemstelling

Op basis van de voorgaande analyse, waaronder observaties en feedback van de developers binnen het bedrijf, kunnen de belangrijkste problemen binnen Devinity als volgt worden samengevat:

- De huidige omgeving maakt gebruik van verschillende tools voor monitoring, logging, security en incidentopvolging, maar deze zijn onvoldoende geïntegreerd. Dit leidt tot een gebrek aan overzicht en maakt het moeilijk om een volledig beeld te krijgen bij incidenten of verdachte activiteiten. Logging en monitoring worden bovendien nog niet systematisch uitgevoerd, waardoor potentiële risico's vaak onopgemerkt blijven.
- De configuratie van de firewall is complex en inconsistent, waardoor het moeilijk is om een efficiënte en veilige configuratie te garanderen. Dit brengt zowel securityrisico's als functionele risico's met zich mee.
- De huidige aanpak van incident response is voornamelijk reactief en er ontbreekt een duidelijke, gestandaardiseerde werkwijze voor het behandelen en opvolgen van incidenten.
- *Door deze factoren is het moeilijk om snel en efficiënt te reageren op security-problemen, en bestaat het risico dat belangrijke signalen niet tijdig worden opgemerkt of correct geïnterpreteerd.* Dit benadrukt de nood aan een geïntegreerde en gestructureerde aanpak voor security monitoring en incident response.



Figuur 4.1: Schematische voorstelling van de versnippering van tools en gebrek aan centralisatie binnen de huidige infrastructuur van Devinity.

4.3.1. Conclusie

De analyse toont duidelijk aan dat er binnen Devinity behoefte is aan een meer gestructureerde en geïntegreerde aanpak voor security monitoring en incident response. De huidige verspreiding van tools, het ontbreken van systematische logging en monitoring, de complexiteit van firewallconfiguraties en de reactieve incident response zijn enkele duidelijke factoren die de noodzaak voor verbeteringen benadrukken.

Een centraal systeem dat logs verzamelt en overzichtelijk presenteert, gecombineerd met geautomatiseerde detectie en respons, kan een groot deel van deze knelpunten aanpakken. Het systeem hoeft niet alle acties automatisch uit te voeren, maar moet vooral zorgen voor duidelijk inzicht, overzicht en een consistente opvolging van incidenten, zodat het hele proces binnen het bedrijf gestroomlijnd wordt en de efficiëntie en helderheid voor het team verbeteren.

Deze probleemanalyse vormt daarmee de basis voor het ontwerp van een oplossing die beter aansluit bij de specifieke behoeften van Devinity en die schaalbaar is binnen hun DevOps-omgeving, zodat monitoring, incident response en securitybeheer efficiënter en betrouwbaarder kunnen verlopen.

Daarbij wordt nadrukkelijk gekozen voor een aanpak die de bestaande infrastructuur en werkwijze respecteert: het doel is niet om alles te vervangen, maar om op een zo min mogelijk ingrijpende manier te bouwen op wat er al aanwezig is. Zo blijft de oplossing realistisch implementeerbaar binnen de DevOps-omgeving van Devinity en wordt de dagelijkse werking van het team zo weinig mogelijk verstoord.

5

Fase 2: Architectuurontwerp

5.1. Inleiding

Op basis van de probleemanalyse wordt in deze fase een concrete architectuur uitgewerkt die toelaat om securitydata binnen de SaaS- en DevOps-omgeving van Devinity op een gestructureerde manier te centraliseren, verwerken en analyseren. Uit de analyse bleek dat de huidige situatie gekenmerkt wordt door een verspreiding van tools zonder centrale coördinatie, een gebrek aan overzicht over actieve dreigingen en een voornamelijk reactieve aanpak van incident response. Incidenten worden met andere woorden vaak pas opgepikt nadat ze zich al hebben voorgedaan, in plaats van proactief gedetecteerd te worden.

Het voorgestelde ontwerp vertrekt vanuit de kernprincipes van SIEM-systemen, namelijk normalisatie, eventcorrelatie en rapportering. Tegelijk blijft het bewust binnen de scope van een proof-of-concept: het doel is niet om een volledig SIEM-platform te bouwen, maar om deze principes toe te passen binnen een haalbare en afgebakende context die aansluit bij de bestaande infrastructuur van Devinity. Omwille van technische haalbaarheid wordt de implementatie daarbij voornamelijk uitgewerkt rond de Azure Web Application Firewall als databron, wat verder wordt toegelicht in de bespreking van de afzonderlijke lagen.

5.2. Architectuuroverzicht

De architectuur is opgebouwd uit drie functionele lagen, aangevuld met een component voor incident response. Elke laag heeft een duidelijk afgebakende verantwoordelijkheid, waardoor de oplossing overzichtelijk blijft en beter aansluit op de

huidige werking van Devinity. Door deze gelaagde opbouw wordt de verwerking van securitydata opgesplitst in duidelijke stappen, gaande van ruwe databronnen over initiële filtering tot centrale analyse. Op deze manier wordt vermeden dat alle verwerking ongecontroleerd en verspreid gebeurt, zoals in de huidige situatie het geval is. Tegelijk blijft de data wel centraal beschikbaar in het analyseplatform, waardoor een globaal overzicht en correlatie van events mogelijk wordt.

De eerste laag omvat de databronnen en blijft ongewijzigd ten opzichte van de huidige situatie. De tweede laag staat in voor data-ingestie, filtering en verwerking, waarbij Make wordt ingezet als centraal automatiseringstool dat de incident response aanstuurt. De derde laag vormt het verwerkingsplatform zelf, gebaseerd op een oplossing die steunt op technologieën binnen Azure, waar data wordt opgeslagen, genormaliseerd, geanalyseerd en gevisualiseerd. Bovenop deze drie lagen bevindt zich de incident response component, die acties triggert op basis van de signalen die Make doorstuurt vanuit laag 2.

Deze gelaagde aanpak maakt het mogelijk om de bestaande infrastructuur zo veel mogelijk te behouden, terwijl tegelijk een meer gecentraliseerde en proactieve monitoringoplossing wordt geïntroduceerd. De scheiding van verantwoordelijkheden zorgt er voor dat wijzigingen in één laag slechts een beperkte impact hebben op de andere lagen.

5.3. Laag 1: Databronnen

De eerste laag bestaat uit alle systemen die securityrelevante data genereren binnen de huidige omgeving van Devinity. Deze bronnen blijven buiten de scope van de voorgestelde aanpassingen, aangezien uit de probleemanalyse bleek vooral dat de verwerking, centralisatie en opvolging van die data het probleem is, en niet noodzakelijk het bestaan van de databronnen zelf.

New Relic levert observabilitydata zoals logs, metrics en alerts die nuttig zijn om afwijkend gedrag, foutpatronen of plotselinge wijzigingen in applicatiegedrag te detecteren. Belangrijk hierbij is echter dat New Relic in de huidige opzet vooral fungeert als eindstation voor applicatie- en codelogs, en niet primair gericht is op security monitoring, waardoor deze niet behandeld zal worden binnen dit proof-of-concept. De Azure Web Application Firewall (WAF) registreert inkomend webverkeer en blokkeringsacties op basis van managed en custom rules. Dit is bijzonder relevant voor het detecteren van aanvallen of verdachte verzoeken, zeker in een context waar GraphQL-verkeer en interne API's een belangrijke rol spelen. Naast de firewalllogs levert de Azure WAF ook aanvullende toegangslogs die informatie bevatten over alle HTTP-verzoeken die de Application Gateway verwerkt, inclusief de bijbehorende statuscodes.

Binnen dit proof-of-concept wordt dan ook bewust gekozen om codelogs en applicatielogs buiten de architectuur te houden. Deze logs zijn immers voornamelijk gericht op applicatieprestaties en foutopsporing, en niet op security monitoring. Het opnemen van deze bronnen zou de scope aanzienlijk verbreden zonder een directe meerwaarde te bieden voor de centrale doelstelling van dit ontwerp, namelijk het centraliseren en opvolgen van securityrelevante data. De oplossing richt zich uitsluitend op security-specifieke logs en alerts, zoals WAF-blokkeringen en beveiligingsgerelateerde events.

Voor dit proof-of-concept wordt de implementatie uitgewerkt rond de Azure WAF als databron, meer bepaald de firewalllogs en toegangslogs die via de Application Gateway worden gegenereerd. Een belangrijke overweging hierbij is dat er momenteel veel logs terechtkomen in New Relic, die echter niet gericht zijn op security maar op applicatie- en codelogs. De Azure WAF levert daarentegen data op die direct relevant is voor security monitoring. Daarnaast is New Relic een externe tool waarvan het gebruik op termijn kan wijzigen, waardoor een directe afhankelijkheid ervan in de ingestiepipeline bewust wordt vermeden. De architectuur is zo opgezet dat andere bronnen in een latere fase op een gelijkaardige manier kunnen worden geïntegreerd, waarbij telkens een tussenlaag zorgt voor consistente verwerking en doorstuur naar het centrale platform.

Naast de Azure WAF wordt ook Azure Service Health opgenomen als databron, op expliciete vraag van de developers tijdens de probleemanalyse. Azure Service Health biedt inzicht in de beschikbaarheid en gezondheid van Azure-diensten die relevant zijn voor de infrastructuur van Devinity, en genereert meldingen bij service-incidenten, geplande onderhoudsmomenten en informatieve statusupdates. Hoewel deze meldingen niet rechtstreeks betrekking hebben op aanvalsdetectie, zijn ze operationeel relevant: een service-incident of gepland onderhoudsvenster kan de werking van de WAF of andere beveiligingscomponenten beïnvloeden en dient tijdig gecommuniceerd te worden naar de betrokken developers.

5.4. Laag 2: Data-ingestie, filtering en verwerking

De tweede laag fungeert als brug tussen de databron en het centrale platform. Azure Monitor stuurt via een webhook een notificatie naar Make wanneer een alert wordt getriggerd, waarna Make de relevante logdata ophaalt via de Log Analytics API en verwerkt voor verdere opslag en opvolging. Het doel is niet om al volledige analyse uit te voeren, maar om relevante events op een gecontroleerde manier op te vangen, door te sturen en waar nodig een incident response te triggeren.

Binnen deze architectuur wordt Make¹ gebruikt als centrale ingestietool. Deze

¹Make (voorheen Integromat) is een visueel automatiseringsplatform waarmee workflows kunnen

keuze is onderbouwd vanuit verschillende invalshoeken: de bestaande werking van Devinity, een vergelijking met alternatieven, en de toekomstgerichtheid van de oplossing.

Voor de verwerking van Azure Service Health alerts wordt een afzonderlijk Make-scenario opgezet, los van het WAF-verwerkingsscenario. Deze scheiding is een bewuste ontwerpkeuze. Service Health alerts zijn fundamenteel anders opgesteld dan WAF-security-events en vereisen een eigen routinglogica en afhandelingsstrategie. Bovendien worden Service Health alerts, in tegenstelling tot WAF-events, niet doorgestuurd naar het centrale platform in laag 3. Opname in Azure Log Analytics zou weinig meerwaarde bieden, omdat het hier gaat om operationele statusmeldingen van Azure-diensten en niet om securityrelevante data die correlatie of historische analyse vereist. In plaats daarvan worden deze alerts rechtstreeks vanuit laag 2 doorgestuurd naar de incident response component, waar ze worden afgehandeld via Monday.com en Outlook op basis van het type melding. Dit houdt het centrale platform gefocust op securitydata en vermijdt dat het wordt overbelast met operationele ruis.

5.4.1. Aansluiting bij de bestaande werking.

Make wordt binnen Devinity al actief ingezet voor het automatiseren van workflows en het doorsturen van meldingen. Het uitbreiden van deze bestaande Make-omgeving met een meer structurele ingestierol is daarom aanzienlijk minder ingrijpend dan het introduceren van een volledig nieuwe tool. De developers zijn vertrouwd met de werking van Make, bestaande scenario's kunnen worden gemakkelijk hergebruikt of uitgebreid, en er is geen bijkomende leercurve of onboarding nodig. Dit sluit direct aan bij de doelstelling om de huidige werkwijze zo weinig mogelijk te verstoren.

5.4.2. Vergelijking met alternatieven.

Een voor de hand liggend alternatief zou zijn om de WAF-diagnostics rechtstreeks te koppelen aan Azure Log Analytics, zonder tussenlaag. Azure biedt hiervoor native ondersteuning via diagnostische instellingen. Deze aanpak is technisch eenvoudig, maar biedt weinig flexibiliteit: alle ruwe logdata wordt onbewerkt doorgestuurd, zonder mogelijkheid tot selectie of filtering vóór opslag. Dit leidt tot hogere opslagkosten en meer ruis in het analyseplatform.

Een bijkomend nadeel van de native aanpak is dat ze stopt bij de opslag van data. Er is geen ingebouwde koppeling met externe tools voor incident response. Make daarentegen werkt niet alleen als ingestietool, maar ook als aansturing: op basis

worden opgebouwd zonder code te schrijven.

van de ernst van een event worden automatisch acties ondernomen. Zo kunnen beveiligingsincidenten rechtstreeks vertaald worden naar Outlook e-mails of werkitens in Monday.com.

Make biedt daarentegen een gemakkelijke, visueel overzichtelijke omgeving die meerdere externe en interne bronnen kan aanspreken via webhooks, API-polling en andere connectoren. Dit maakt het bijzonder geschikt als ingestIELaag in een omgeving waar de databronnen divers zijn en niet allemaal binnen hetzelfde ecosysteem zitten.

5.4.3. Toekomstgerichtheid en uitbreidbaarheid.

Een belangrijk voordeel van Make als ingestietool is de schaalbaarheid naar de toekomst toe. De huidige implementatie focust op de Azure WAF als primaire bron, maar de architectuur is bewust generiek opgezet. Wanneer in een latere fase bijkomende bronnen worden geïntegreerd, zoals New Relic-alerts, meldingen vanuit Microsoft 365 Defender of events uit externe SaaS-integraties, kan dit telkens worden gerealiseerd door een nieuw scenario toe te voegen binnen de bestaande Make-omgeving. Er is geen aanpassing nodig aan het centrale platform of de incident response component. Make fungeert zo als een flexibele en uitbreidbare toegangspoort tot het centrale analyseplatform, waarbij elke nieuwe databron op een vergelijkbare en gestandaardiseerde manier wordt opgenomen. Dit maakt de oplossing niet alleen waardevol binnen de huidige scope, maar ook duurzaam inzetbaar naarmate de infrastructuur van Devinity verder groeit.

Concreet worden in dit proof-of-concept scenario's opgezet die logdata en alerts binnenhalen vanuit de Azure WAF als primaire bron. De opzet blijft evenwel generiek genoeg om in een latere fase andere bronnen op een vergelijkbare manier te integreren. Afhankelijk van de technische mogelijkheden van elke bron gebeurt dit via een webhook, een connector, een periodieke API-opvraag of het uitlezen van inkomende notificaties. Zodra een event wordt ontvangen, wordt het geformatteerd en voorzien van de nodige context zodat het doorgegeven kan worden aan het centrale platform.

5.4.4. Initiële filtering en selectie

Niet alle data uit de bronnen is even relevant voor security monitoring. Daarom zal in deze laag reeds een eerste filtering worden toegepast, met als doel ruis te beperken en enkel de events door te sturen die waardevol zijn voor verdere analyse. Denk bijvoorbeeld aan WAF-blokkeringen die matchen met bekende aanvalspatronen of meldingen die wijzen op afwijkend verkeer.

Waarom filteren in Make en niet in Log Analytics?

Een bewuste ontwerpkeuze is om de initiële filtering te laten plaatsvinden in Make, en niet pas na opslag in Azure Log Analytics. Hoewel Log Analytics krachtige query-mogelijkheden biedt via de Kusto Query Language (KQL), heeft het doorsturen van alle ruwe data zonder voorafgaande selectie twee nadelen. Ten eerste verhoogt het de opslagkost, omdat ook irrelevante events worden bewaard. Ten tweede vergroot het de hoeveelheid ruis in het analyseplatform, wat het herkennen van relevante patronen bemoeilijkt.

De bedoeling is niet om in Make al uitgebreide correlatie of diepgaande analyse uit te voeren, maar om de datastroom te structureren en het centrale platform te ontlasten. De inhoudelijke verwerking en correlatie vindt plaats in de derde laag.

5.5. Laag 3: Centraal platform voor verwerking en analyse

De derde laag vormt de kern van de architectuur en is verantwoordelijk voor de effectieve verwerking van de gefilterde securitydata. Alle relevante events worden hier centraal opgeslagen, genormaliseerd, geanalyseerd en gevisualiseerd. Dit is de laag die het huidige gebrek aan overzicht en centralisatie moet oplossen.

Voor de implementatie van dit centrale platform wordt gekozen voor een Azure-gebaseerde oplossing, meer bepaald Azure Log Analytics in combinatie met Azure Monitor. Deze keuze volgt logisch uit zowel de bestaande infrastructuur van Devinity als een aantal technische overwegingen.

5.5.1. Aansluiting bij de bestaande infrastructuur.

Devinity maakt reeds gebruik van Azure voor infrastructuur en beveiliging, en meer bepaald voor de Web Application Firewall die als primaire databron fungeert in dit proof-of-concept. Dat betekent dat de WAF-logs zich al binnen het Azure-ecosysteem bevinden. Een externe analysetool zou deze data eerst moeten exporteren naar een ander platform, wat een onnodige extra stap introduceert in de pipeline, het aanvalsoppervlak vergroot en de kans op dataverlies of vertraging verhoogt. Door te kiezen voor Azure blijft de volledige datastroom binnen dezelfde omgeving. Dit vermijdt onnodige exports naar externe platforms, verkleint het aanvalsoppervlak en verlaagt de kans op dataverlies of vertraging in de pipeline, wat rechtstreeks aansluit bij de doelstelling om de bestaande infrastructuur van Devinity zo weinig mogelijk te verstoren.

5.5.2. Kostenbeheer en licenties.

Het gebruik van Azure Log Analytics valt binnen het bestaande Azure-abonnement van Devinity, zonder dat een bijkomende licentie voor een volledig SIEM-platform vereist is. Commerciële alternatieven brengen aanzienlijk wat kosten met zich mee. De keuze voor een Azure-native oplossing maakt het mogelijk om dezelfde analytische functionaliteiten te benutten tegen een aanzienlijk lagere totale kost.

5.5.3. Veiligheid en data governance.

De securitydata wordt opgeslagen binnen de beveiligde Azure-omgeving van Devinity. Make fungeert als tussenlaag voor ingestie en filtering, wat betekent dat data tijdelijk buiten Azure wordt verwerkt. Binnen de context van dit proof-of-concept is dit een aanvaardbare afweging. Enkel de relevante velden worden doorgestuurd, de volledige ruwe logdata blijft bewaard in Azure, en de toegang tot Make wordt beheerd via een service principal met beperkte rechten.

5.5.4. Schaalbaarheid en toekomstbestendigheid.

Een Azure-gebaseerde oplossing biedt een duidelijk groeipad. Wanneer in een latere fase bijkomende databronnen worden geïntegreerd, zoals New Relic-alerts of meldingen vanuit externe applicaties, kan dit gebeuren binnen dezelfde omgeving zonder dat een platformmigratie nodig is. De architectuur groeit zo mee met de infrastructuur van Devinity, zonder dat er een breuk ontstaat in de werkwijze of toolstack.

Belangrijk hierbij is dat deze keuze niet bedoeld is als vervanging van alle bestaande tools. De WAF en andere diensten blijven databronnen, terwijl het centrale platform uitsluitend de rol van verwerking, analyse en rapportering opneemt.

5.5.5. Analyse en correlatie

Na normalisatie kunnen de events worden geanalyseerd via KQL-query's binnen het centrale platform. Hierdoor kunnen patronen worden herkend die in de huidige situatie moeilijk zichtbaar zijn, net omdat de data verspreid zit over verschillende tools. Een combinatie van meerdere gebeurtenissen kan wijzen op een relevant risico dat op basis van afzonderlijke meldingen niet zichtbaar zou zijn. Binnen de scope van dit proof-of-concept wordt de analyse beperkt tot filtering en aggregatie per databron. Correlatie tussen events uit verschillende bronnen vormt een aanbeveling voor een volgende fase.

5.5.6. Visualisatie en rapportering

Naast analyse moet het platform ook inzicht bieden aan de betrokkenen binnen Devinity. Daarom wordt een visualisatielaag voorzien in de vorm van dashboards en overzichten. Deze dashboards laten toe om op een overzichtelijke manier de actuele securitystatus op te volgen, trends te herkennen en gebeurtenissen achteraf te analyseren.

Een bijkomend voordeel van centrale opslag is dat historische data beschikbaar blijft voor latere rapportering en evaluatie. Dit maakt het mogelijk om niet enkel incidenten op te volgen, maar ook KPI's te definiëren en de effectiviteit van de monitoring op termijn te beoordelen.

5.6. Incident response

Op basis van de signalen die Make verwerkt in laag 2, kunnen acties worden ondernomen in het kader van incident response. Make fungeert hierbij niet alleen als ingestietool maar ook als triggerlaag voor geautomatiseerde opvolging. Wanneer een binnenkomend event aan de filtervoorwaarden voldoet, kan Make automatisch een notificatie naar de betrokken personen via e-mail sturen of een workitem aanmaken binnen Monday.com.

Deze aanpak sluit beter aan bij de noden uit de probleemanalyse dan de huidige reactieve werking. Vandaag worden meldingen wel geregistreerd, maar vaak niet systematisch opgevolgd. Door incident response rechtstreeks te koppelen aan de ingestielaag kan sneller worden gereageerd en wordt de kwaliteit van de opvolging verbeterd. Monday.com wordt daarnaast al actief gebruikt binnen Devinity, waardoor de drempel voor adoptie minimaal is.

5.7. Evaluatie ten opzichte van de doelstellingen

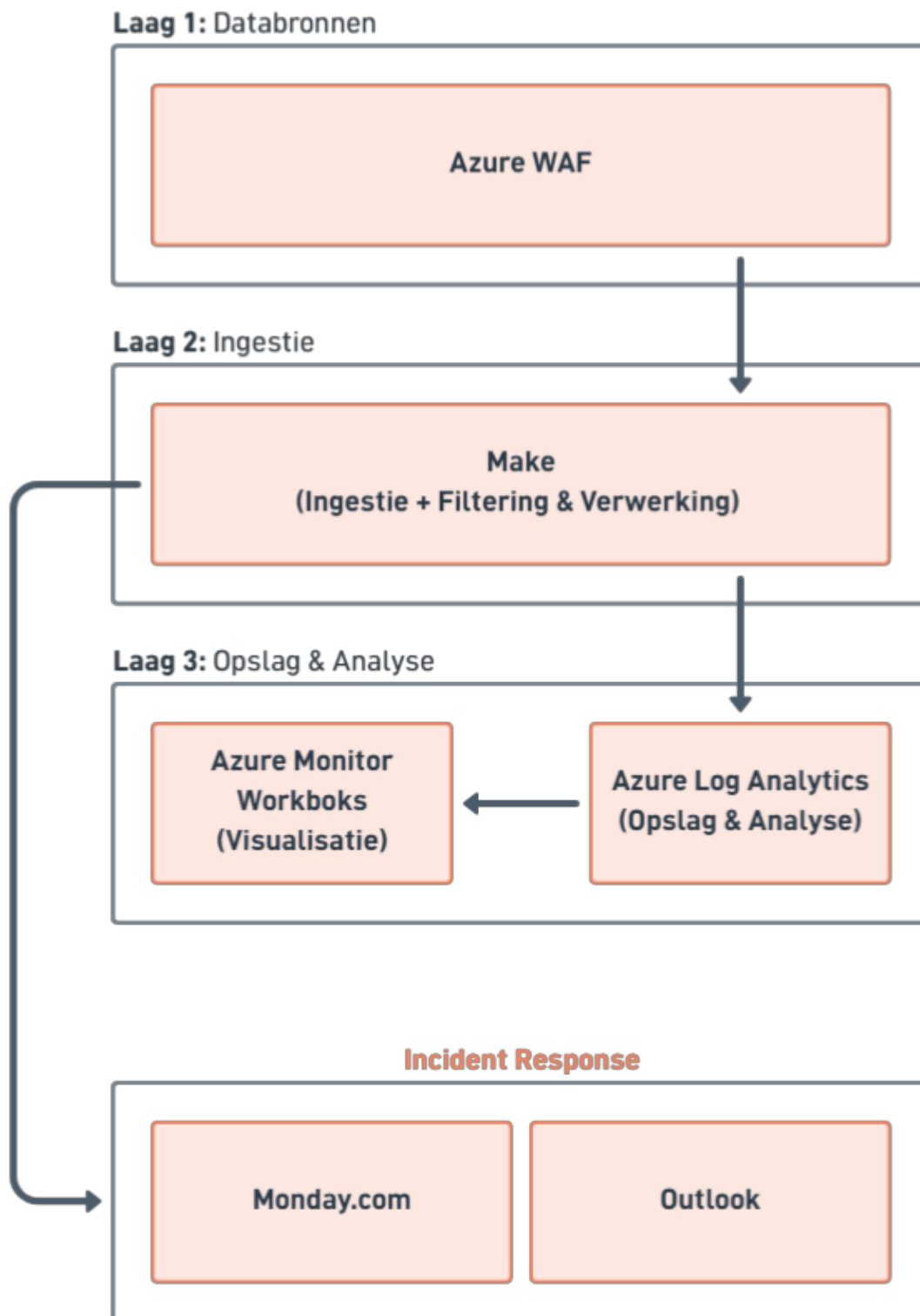
De voorgestelde architectuur sluit aan bij de doelstellingen zoals geformuleerd in de inleiding en de onderzoeksvragen. De centralisatie van securitydata wordt gerealiseerd via een centraal platform dat gevoed wordt door een gestandaardiseerde ingestiepipeline met Make. De detectie van incidenten wordt mogelijk gemaakt door de combinatie van initiële filtering in laag 2 en correlatie in laag 3. Gestructureerde incident response wordt ondersteund via automatische notificaties en workitems die rechtstreeks vanuit Make worden getriggerd.

De oplossing blijft bovendien haalbaar binnen de context van de bachelorproef, omdat zoveel mogelijk gebruik wordt gemaakt van bestaande tools en infrastructuur. Er is bewust geen volledige vervanging van de huidige omgeving voorzien,

maar eerder een gerichte integratie van de onderdelen die vandaag al beschikbaar zijn. De keuze om het proof-of-concept te centreren rond WAF-logdata vormt geen beperking voor de bredere probleemstelling. De architectuur is generiek uitgewerkt en de bevindingen zijn overdraagbaar naar een bredere integratie van de overige databronnen die in de probleemanalyse werden geïdentificeerd.

De effectiviteit van het systeem kan in een latere fase worden geëvalueerd aan de hand van KPI's zoals detectiesnelheid, responstijd, het aantal relevante versus irrelevante meldingen en de mate waarin incidenten vollediger kunnen worden opgevolgd.

5.8. Visuele voorstelling van de architectuur



5.9. Conclusie

De voorgestelde architectuur biedt een gestructureerde en haalbare oplossing voor de uitdagingen die in de probleemanalyse werden geïdentificeerd. Door te werken met drie duidelijk gescheiden lagen en een aparte incident response component wordt de versnippering van data aangepakt en wordt een basis gelegd voor een meer proactieve aanpak van security monitoring.

De keuze om gebruik te maken van reeds beschikbare tools (Make voor ingestie, filtering en verwerken van meldingen, een Azure-gebaseerd platform voor verwerking en opslag, en Monday.com in combinatie met Outlook voor opvolging en notificatie) maakt de oplossing realistisch binnen de context van Devinity. Er wordt bewust geen volledig nieuw platform van nul opgebouwd, maar een geïntegreerde architectuur ontworpen die de bestaande omgeving beter benut en centrale inzichten mogelijk maakt.

In de volgende fase wordt deze architectuur verder uitgewerkt tot een concreet proof-of-concept, waarbij de verschillende componenten worden geconfigureerd en getest aan de hand van realistische scenario's.

6

Fase 3: Proof-of-concept ontwikkeling

6.1. Inleiding

Op basis van het architectuurontwerp uit de vorige fase wordt in dit hoofdstuk het effectieve proof-of-concept uitgewerkt. Het doel is niet om een productierijp systeem te bouwen, maar om de architectuurprincipes concreet te valideren aan de hand van een werkend prototype. De kernfuncties uit het ontwerp worden daarvoor stapsgewijs geïmplementeerd: logverzameling vanuit de Azure WAF, initiële filtering en verwerking via Make, centrale opslag en analyse in Azure Log Analytics, visualisatie via Azure Monitor Workbooks en geautomatiseerde incident response via Monday.com en Outlook.

De implementatie verloopt in een testomgeving binnen de Azure-portal van het bedrijf en maakt gebruik van gesimuleerde data en ongevoelige logdata om realistische scenario's na te bootsen zonder afhankelijk te zijn van productiedata. Dit laat toe om de werking van het systeem te valideren op een gecontroleerde en herhaalbare manier.

6.2. Testdata en simulatiescenario's

Voordat de technische implementatie wordt beschreven, is het nuttig om stil te staan bij de data die gebruikt wordt om het systeem te testen. Omdat het proof-of-concept werkt binnen een testomgeving, worden twee soorten data ingezet: enerzijds gesimuleerde events die doelbewust worden gegenereerd om specifieke sce-

nario's na te bootsen, anderzijds ongevoelige logdata uit bestaande testomgevingen die een realistisch beeld geven van het normale verkeerspatroon.

De gesimuleerde testdata omvat onder meer WAF-blokkingsevents die overeenkomen met gekende aanvalspatronen zoals SQL-injectie of cross-site scripting (XSS), gegenereerd via gerichte HTTP-verzoeken naar de Application Gateway. De gateway blokkeert deze verzoeken met een HTTP-antwoord en registreert het bijbehorende event in Log Analytics. Op die manier kan worden nagegaan of het systeem de juiste detectieregels triggert en of de verdere verwerking correct verloopt.

Naast de WAF-blokkingsevents worden ook scenario's gesimuleerd die de **Security-Error-Alert** triggeren. Hiervoor worden via een geautomatiseerd curl-script herhaalde HTTP-verzoeken verstuurd naar de Application Gateway die resulteren in specifieke statuscodes zoals 401, 403 en 404. Door hetzelfde IP-adres meer dan twintig keer dezelfde statuscode te laten genereren binnen een tijdsvenster van vijf minuten, wordt de drempelwaarde van de Alert Rule bewust overschreden. Dit laat toe om te valideren of de filterlogica correct werkt en of de verdere verwerking via Make, Log Analytics en de incident response-modules zich gedraagt zoals verwacht.

6.3. Laag 1: Databron

De implementatie van de eerste laag bestaat uit het configureren van de Azure WAF als primaire databron, zoals vermeld in het architectuurontwerp. De WAF is geconfigureerd om diagnostische logs door te sturen naar een Azure Log Analytics Workspace via de diagnostische instellingen in Azure Monitor. Hierbij worden de logcategorieën `ApplicationGatewayAccessLog` en `ApplicationGatewayFirewallLog` ingeschakeld, die respectievelijk de toegangslogs en de firewalllogs van de Application Gateway bevatten.

In de gebruikte Azure-omgeving wordt gebruikgemaakt van de nieuwere resource-specifieke logtabellen in plaats van de algemene `AzureDiagnostics`-tabel. Dit onderscheid is relevant omdat resource-specifieke tabellen een vastere schemastructuur hebben, wat KQL-queries eenvoudiger en efficiënter maakt. Concreet betekent dit dat de firewalllogs terechtkomen in de tabel `AGWFirewallLogs` en de toegangslogs in `AGWAccessLogs`. De `AGWFirewallLogs`-tabel bevat onder meer informatie over geblokkeerde verzoeken, de bijbehorende regelidentificatoren en het bron-IP-adres, wat ze bijzonder relevant maakt voor het detecteren van aanvalspatronen. De `AGWAccessLogs`-tabel bevat daarnaast informatie over alle HTTP-verzoeken die de Application Gateway verwerkt, inclusief de bijbehorende statuscodes, clientinformatie en verwerkingstijden. Deze tabel vormt de basis voor de **Security-Error-Alert**, die afwijkende statuscodepatronen detecteert als indica-

tie van mogelijke toegangsproblemen of verdacht gedrag.

De Azure WAF op de Application Gateway maakt gebruik van managed rulesets die door Microsoft worden beheerd en automatisch worden bijgewerkt. Binnen de huidige configuratie zijn twee rulesets actief: de OWASP Core Rule Set (CRS), die bescherming biedt tegen veelvoorkomende webapplicatieaanvallen zoals SQL-injectie, cross-site scripting en path traversal, en de Microsoft *BotManagerRuleSet*, die bekend is om automatische detectie van bots. De *BotManagerRuleSet* is een managed ruleset die wordt bijgewerkt door Microsoft en die is ontworpen om bots te detecteren en te blokkeren. Het is een van de meest geavanceerde rulesets die beschikbaar is voor de Azure WAF. Het is een van de meest geavanceerde rulesets die beschikbaar is voor de Azure WAF. Het is een van de meest geavanceerde rulesets die beschikbaar is voor de Azure WAF.

Omdat het verwerkingssysteem de events op basis van generieke velden uit de AGWFirewallLogs-tabel verwerkt, zoals RuleId, RuleGroup, Action en Message, is de architectuur niet gebonden aan een specifieke ruleset. Bijkomende managed rulesets die Microsoft in de toekomst beschikbaar stelt, zoals de Microsoft *DefaultRuleSet* of aanvullende regels voor DDoS-bescherming, kunnen worden geactiveerd op de Application Gateway zonder aanpassingen aan de verwerking van de events. Het is een van de meest geavanceerde rulesets die beschikbaar is voor de Azure WAF. Het is een van de meest geavanceerde rulesets die beschikbaar is voor de Azure WAF. Het is een van de meest geavanceerde rulesets die beschikbaar is voor de Azure WAF.

Beide tabellen worden enkel ingeschakeld voor de relevante logcategorieën. Andere diagnostische logcategorieën zoals ApplicationGatewayPerformanceLog worden bewust buiten de configuratie gehouden, omdat deze geen securityrelevante data bevatten en het volume aan opgeslagen data onnodig zouden verhogen.

6.4. Laag 2: Data-ingestie, filtering en verwerking via Make

6.4.1. Koppeling tussen Azure Monitor en Make

Zoals beschreven in het architectuurontwerp fungeert Make als centrale tussenlaag tussen de databron en het centrale platform. Om WAF-events automatisch naar Make te sturen, wordt een Azure Monitor Alert Rule geconfigureerd met een bijbehorende Action Group. De Alert Rule is gekoppeld aan de Log Analytics Workspace als scope en maakt gebruik van een KQL-query¹ op de AGWFirewallLogs-tabel om geblokkeerde events rechtstreeks van de WAF te detecteren:

```

1 AGWFirewallLogs
2 | where TimeGenerated >= ago(5m)
3 | where Action == "Blocked"
4 | summarize count()

```

Listing 6.1: KQL-query voor de Azure Monitor Alert Rule op geblokkeerde WAF-events

Deze query selecteert alle geblokkeerde events uit de afgelopen vijf minuten en telt deze op, zodat de alert triggert zodra er minstens één geblokkeerd event is geregistreerd. De bijbehorende Action Group bevat één webhook-actie die bij elke

¹Kusto Query Language (KQL) is de querytaal die gebruikt wordt binnen Azure Log Analytics en Azure Monitor.

triggering een HTTP POST stuurt naar het Make webhook-endpoint. Er wordt bewust gekozen voor één enkele webhook-actie per trigger, omdat Make werkt met een maandelijks kredietlimiet. Elke uitvoering van een scenario verbruikt credits, waardoor een efficiënte opzet waarbij de verwerking zoveel mogelijk gebundeld wordt binnen één scenario de voorkeur krijgt boven meerdere afzonderlijke triggers.

Naast de firewall-gerelateerde alerts wordt een tweede Alert Rule geconfigureerd, Security-Error-Alert, die draait op de AGWAccessLogs-tabel. Deze regel detecteert afwijkende HTTP-statuscodes zoals 401, 403, 404, 429 en 500 en triggert wanneer een combinatie van IP-adres en statuscode minstens twintig keer voorkomt binnen een tijdsvenster van vijf minuten:

```
1 AGWAccessLogs
2 | where TimeGenerated >= ago(5m)
3 | where ServerStatus in (404, 500, 401, 403, 429)
4 | summarize Count = count() by ClientIp, ServerStatus
5 | where Count >= 20
```

Listing 6.2: KQL-query voor de Security-Error-Alert Rule

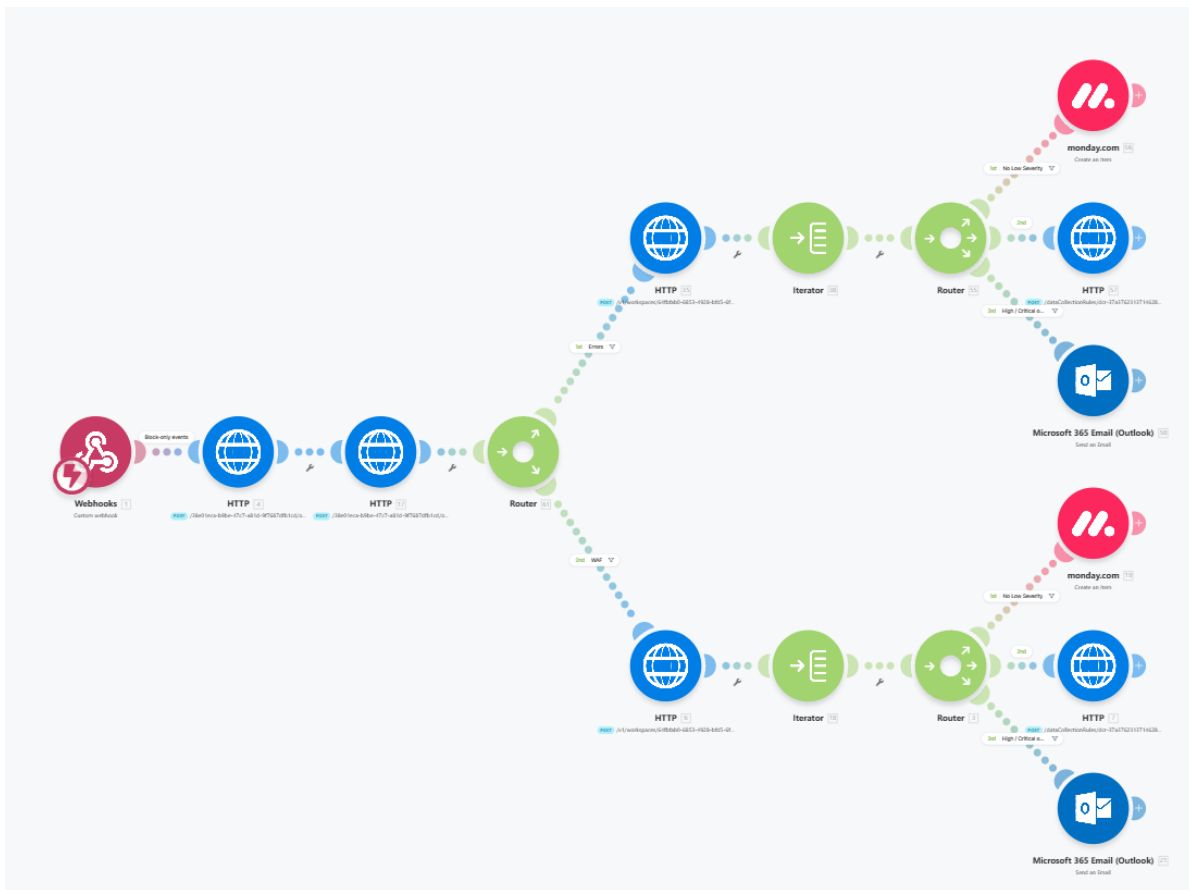
Ook deze Alert Rule is gekoppeld aan dezelfde Action Group, waardoor bij elke triggering eveneens één webhook-actie naar het Make webhook-endpoint wordt verstuurd.

Deze regel biedt ook een indirecte benadering voor het detecteren van mogelijke brute-force of credential-stuffing pogingen. Herhaalde 401- en 403-responses van hetzelfde IP-adres binnen een kort tijdsvenster wijzen namelijk op herhaalde mislukte autorisatiepogingen, wat kan duiden op geautomatiseerde aanvallen gericht op ongeautoriseerde toegang. Omdat Entra ID-logs niet beschikbaar zijn binnen de huidige omgeving, vormt deze aanpak een praktisch haalbaar alternatief voor authenticatiegebaseerde anomaliedetectie.



6.4.2. Make-scenario: opbouw en werking

Binnen Make wordt een scenario opgezet dat de binnenkomende events opvangt en verwerkt. De datastructuur van de webhook werd bepaald door een testpayload te sturen via een curl-commando met een representatief WAF-event in JSON-formaat, zodat Make de veldnamen herkent en het scenario de juiste filtercriteria kan toepassen.



Figuur 6.2: Overzicht van het Make-scenario voor de verwerking van security-events

Het scenario bestaat uit vier opeenvolgende onderdelen: een initiële stap waarin via twee HTTP-modules de nodige tokens worden opgehaald voor toegang tot de Azure Log Analytics Workspace, een opsplitsing op basis van het eventtype, een typespecifieke verwerking per pad gebaseerd op de payload, en een gecombineerde incident response aan het einde van elk pad.

Via de twee eerste HTTP-modules worden tokens opgehaald die nodig zijn om toegang te krijgen tot de Azure Log Analytics Workspace. Eerst wordt via de **Microsoft identity endpoint** een access token gegenereerd op basis van de App Registration. Dit token fungeert als authenticatiebewijs voor verdere API-aanroepen. Vervolgens wordt deze access token gebruikt om geautoriseerde requests uit te voeren richting de Log Analytics API, waar de effectieve query's op de workspace worden uitgevoerd om de relevante security-events op te halen en verder te verwerken binnen het scenario.

6.4.3. Initiële filtering via de eerste Router-module

De eerste Router-module vormt het vertrekpunt van de verwerkingslogica en splitst de binnenkomende events op in twee afzonderlijke paden op basis van het veld `data.essentials.alertRule`. Wanneer dit veld de waarde `Security-FirewallLog-Alert` bevat, wordt het event doorgestuurd via de WAF-route. Bevat het veld de waarde `Security-Error-Alert`, dan volgt het event het pad voor foutmeldingen. Deze opsplitsing is nodig omdat beide types inhoudelijk verschillen en een andere verwerking vereisen: een WAF-firewalllog bevat informatie over geblokkeerde requests en bijhorende regelidentificatoren, terwijl een foutmelding eerder wijst op afwijkende HTTP-statuscodes die kunnen duiden op een technisch probleem of verdacht gedrag. Door dit onderscheid vroeg in het scenario te maken, kan elk pad volledig worden afgestemd op het specifieke eventtype.

The image displays two side-by-side screenshots of the 'Set up a filter' configuration window in the Make platform. Both windows show a filter being configured with the label 'Errors' (left) and 'WAF' (right). The condition is set to '1. data: essentials: alertRule' with the operator 'Text operators: Equal to'. The left window has the value 'Security-Error-Alert' selected, while the right window has 'Security-FirewallLog-Alert' selected. Both filters have 'No' selected for 'Set the route as a fallback'. The 'Save' button is highlighted in purple in both windows.

Figuur 6.3: Filterconfiguratie van de eerste Router-module: opsplitsing tussen WAF-firewalllogs en foutmeldingen op basis van het veld `data.essentials.alertRule`

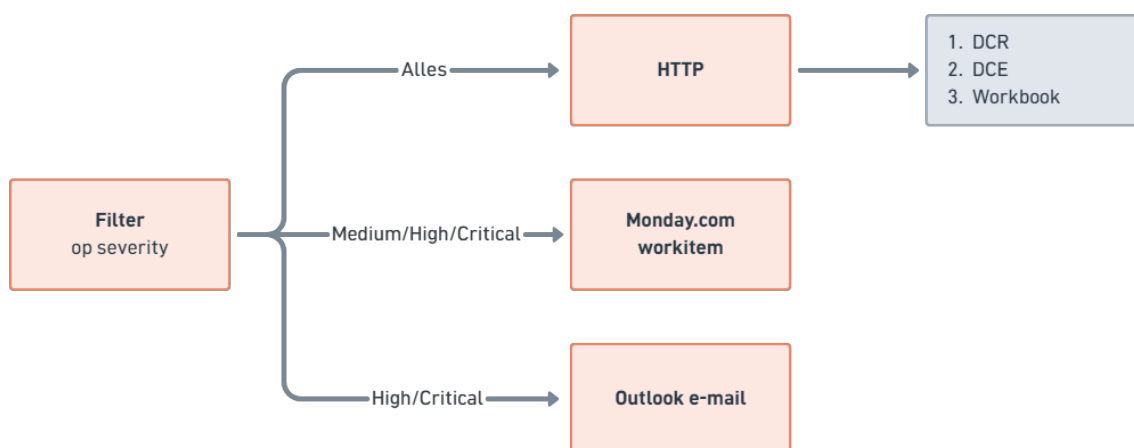
Events die niet overeenkomen met een van beide routes, worden genegeerd en doorlopen geen verdere stappen, wat aansluit bij de ontwerpkeuze uit het architectuurontwerp om de initiële filtering te laten plaatsvinden vóór opslag zodat ruis wordt beperkt en het centrale platform enkel relevante data ontvangt.

6.4.4. Verwerking per pad

Elk pad volgt na de initiële opsplitsing dezelfde structuur. Een HTTP-module haalt aanvullende gegevens op uit de Azure-omgeving via de Log Analytics API. Een Iterator-module verwerkt vervolgens de individuele elementen uit de ontvangen

lijst, waarna een Tools-module verdere transformaties toepast op de data. Een tweede Router-module bepaalt ten slotte de uiteindelijke afhandeling op basis van de inhoud van het event.

Afhankelijk van de severity wordt het event op drie manieren afgehandeld. Alle events worden steeds doorgestuurd via een HTTP-module naar een Data Collection Rule voor verdere verwerking binnen de Azure Log Analytics Workspace en visualisatie in een Workbook. Events met severity medium, high en critical worden daarnaast geregistreerd als workitem in Monday.com. Voor high en critical events wordt bijkomend een e-mailnotificatie verstuurd via Microsoft 365 Outlook naar de betrokken verantwoordelijken.



6.4.5. Severitybepaling

Omdat de managed rulesets binnen de Azure WAF geen native CVSS-severityscore meegeven per event, wordt de ernst van een event bepaald op basis van de beschikbare logdata. Voor events afkomstig van de `Microsoft_BotManagerRuleSet` wordt een vaste severity Low toegekend, omdat deze regelset voornamelijk bekende bots detecteert die zelden een kritiek risico vormen. Voor events afkomstig van de OWASP CRS-regelset wordt de severity afgeleid uit de anomaly score, die terug te vinden is in het Message-veld van de `AGWFirewallLogs`-tabel in het formaat `Total Score: X`. Deze score wordt omgezet naar een severitylabel volgens onderstaande drempelwaarden:

Anomaly Score	Severity
< 5	Low
≥ 5	Medium
≥ 10	High
> 25	Critical

Tabel 6.1: Drempelwaarden voor severitybepaling op basis van OWASP CRS anomaly score

Voor events afkomstig van de `Security-Error-Alert` wordt de severity bepaald op basis van het aantal herhaalde foutieve responses van hetzelfde IP-adres binnen het gedefinieerde tijdsvenster. Hoe hoger het aantal herhalingen, hoe groter de kans op geautomatiseerd of kwaadaardig gedrag. De drempelwaarden zijn als volgt:

Aantal herhalingen	Severity
< 25	Low
≥ 25	Medium
≥ 50	High
≥ 100	Critical

Tabel 6.2: Drempelwaarden voor severitybepaling op basis van herhaalde foutieve responses

Deze drempelwaarden zijn gebaseerd op eigen inschatting binnen de context van dit proof-of-concept en zijn niet afgeleid van een officiële standaard. Ze kunnen in een productieomgeving worden bijgesteld op basis van de normale verkeerspatronen van de applicatie.

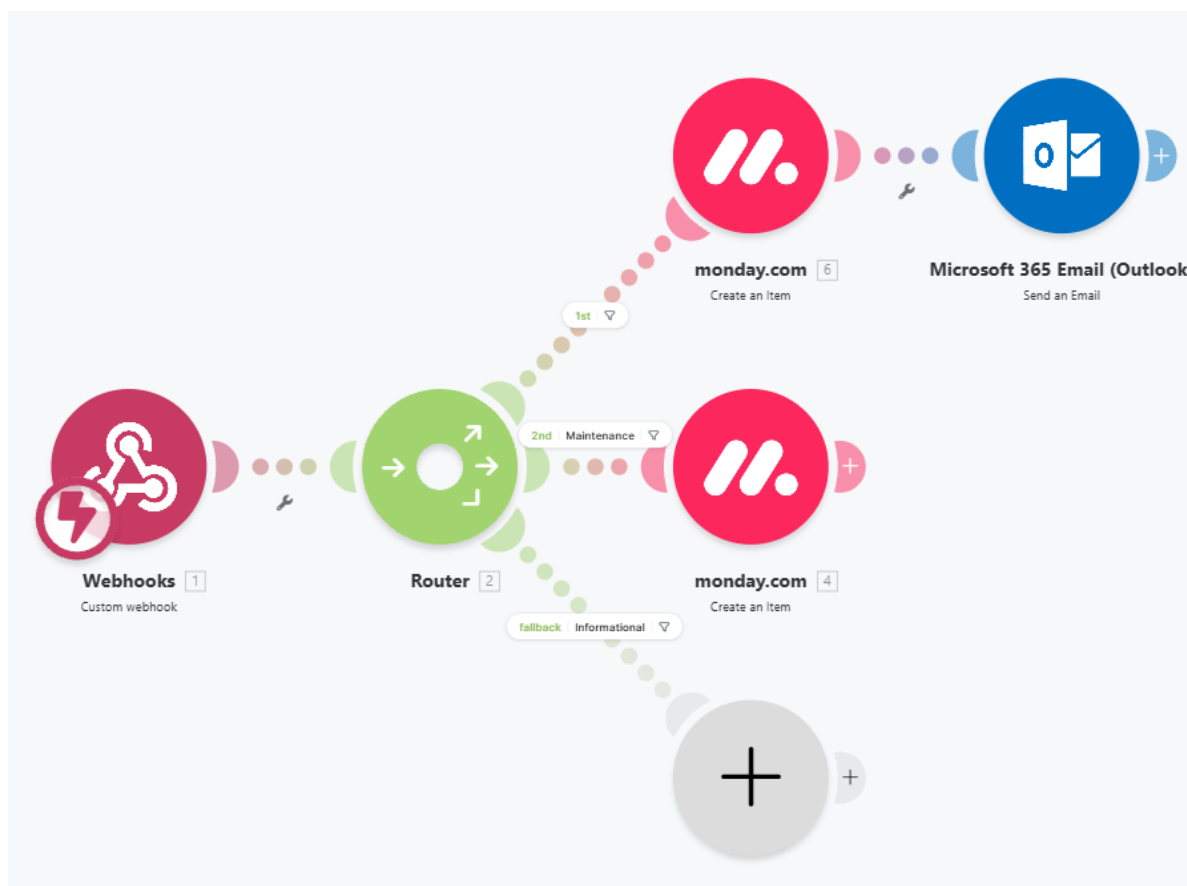
6.4.6. Make-scenario: Azure Service Health alerts

Naast het WAF-verwerkingsscenario wordt een tweede, afzonderlijk Make-scenario opgezet voor de verwerking van Azure Service Health alerts. Dit scenario werd opgenomen op expliciete vraag van de developers tijdens de probleemanalyse, die aangaven nood te hebben aan een gestructureerde opvolging van service-incidenten en onderhoudsmeldingen die betrekking hebben op de Azure-infrastructuur van Devinity.

De koppeling verloopt op dezelfde manier als bij het WAF-scenario: een Azure Monitor Action Group met een webhook-actie stuurt bij elke triggering een HTTP POST naar het Make webhook-endpoint van dit scenario. Binnen Azure Monitor wordt hiervoor een aparte Service Health Alert Rule geconfigureerd die abonneert

op alle relevante Service Health-notificaties binnen het Azure-abonnement van Devinity.

Een belangrijk verschil met het WAF-scenario is dat Service Health alerts niet worden doorgestuurd naar het centrale platform in laag 3. Omdat het hier gaat om operationele statusmeldingen van Azure-diensten en niet om securityrelevante data, biedt opname in Azure Log Analytics weinig meerwaarde. Centrale opslag zou het analyseplatform onnodig belasten met data die geen bijdrage levert aan security-analyse of correlatie. In plaats daarvan worden deze alerts rechtstreeks vanuit Make doorgestuurd naar de incident response component, wat aansluit bij de doelstelling om het centrale platform gefocust te houden op securitydata.



Figuur 6.5: Overzicht van het Make-scenario voor de verwerking van Azure Service Health alerts

Het scenario is opgebouwd rond een centrale Router-module die de binnenkomende melding opsplitst op basis van het type Service Health event. Drie paden worden onderscheiden: Incident, Maintenance en Informational.

Het Incident-pad wordt gevolgd wanneer een actieve verstoring van een Azure-dienst wordt gemeld. Omdat dergelijke incidenten een directe impact kunnen hebben op de beschikbaarheid van de applicatie, wordt automatisch een workitem aangemaakt in Monday.com én een e-mailnotificatie verstuurd via Microsoft

365 Outlook naar de betrokken verantwoordelijken, zodat onmiddellijke opvolging mogelijk is.

Het `Maintenance`-pad wordt geactiveerd bij geplande onderhoudsmomenten die door Microsoft vooraf worden aangekondigd. In dit geval volstaat het aanmaken van een workitem in Monday.com, zodat de developers tijdig op de hoogte zijn van het geplande onderhoudsvenster en hier rekening mee kunnen houden in hun planning. Een e-mailnotificatie is hier minder urgent en wordt dan ook niet verstuurd.

Het `Informational`-pad vangt alle overige informatieve statusupdates op. Omdat deze meldingen geen directe actie vereisen, worden ze binnen de huidige scope van het proof-of-concept niet verder afgehandeld. Dit pad fungeert als fallback voor events die niet als incident of onderhoud worden geclassificeerd.

Deze drieledige opsplitsing zorgt ervoor dat de urgentie van de melding bepalend is voor de mate van opvolging: hoe groter de potentiële impact op de infrastructuur, hoe actiever de notificatie. De volledige afhandeling vindt plaats binnen laag 2, zonder tussenkomst van het centrale platform.

6.5. Laag 3: Centrale verwerking in Azure Log Analytics

6.5.1. Opslag en normalisatie via Data Collection Rule

Gefilterde events worden vanuit Make doorgestuurd naar Azure Log Analytics via de Logs Ingestion API, gebruikmakend van een Data Collection Endpoint (DCE) en een Data Collection Rule (DCR). De DCE werkt als het eindpunt waarnaar Make de data stuurt, terwijl de DCR bepaalt in welke tabel de data terechtkomt en welke transformaties worden toegepast. Deze aanpak sluit aan bij de aanbevelingen van Microsoft voor moderne Azure-integraties en vervangt de HTTP Data Collector API die inmiddels als verouderd wordt beschouwd.

Omdat de doeltabel `WAFFilteredEvents_CL` nog niet bestond in de workspace, werd deze eerst handmatig aangemaakt via de Tables-sectie in de Log Analytics Workspace. Het Analytics-abonnement werd gekozen zodat volledige KQL-querymogelijkheden beschikbaar zijn. Het schema van de tabel omvat de velden `clientId`, `requestUri`, `ruleId`, `ruleGroup`, `action`, `severity` en `message`, wat overeenkomt met de relevante velden uit de WAF-firewalllogs.

Voor de authenticatie van Make richting de Logs Ingestion API wordt gebruikgemaakt van een Azure App Registration. Deze registratie levert een client ID, tenant ID en client secret op waarmee Make via OAuth 2.0 een toegangstoken opvraagt bij Microsoft Entra ID (de eerste 2 HTTP modules). Dit token wordt vervolgens mee-

gestuurd bij elk HTTP-verzoek naar het DCE-eindpunt.

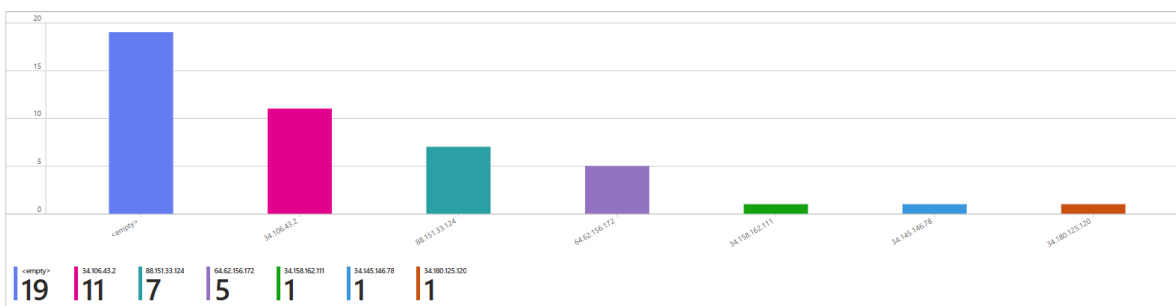
6.6. Visualisatie via Azure Monitor Workbooks

Om de verzamelde en geanalyseerde data inzichtelijk te maken voor de betrokkenen binnen Devinity, worden Azure Monitor Workbooks ingezet als visualisatielaag, zoals voorzien in het architectuurontwerp. De inzichten zijn gebaseerd op query's op de tabel `WAFFilteredEvents_CL`, waardoor de dashboards uitsluitend geaggregeerde en relevante security-events vanuit Make weergeven in plaats van ruwe logdata.

Het workbook is opgebouwd rond een operationeel security-overzicht dat het aantal WAF-events over een instelbare tijdsperiode visualiseert, inclusief trendanalyse in de tijd. Daarnaast worden de meest actieve bron-IP-adressen en de frequentie van blokkeringen weergegeven om aanvallende bronnen te identificeren. Een bijkomend onderdeel focust op de meest voorkomende rule groups, waardoor dominante aanvalspatronen snel zichtbaar worden. Tot slot wordt de spreiding van blokkeringen gevisualiseerd op basis van tijd om piekmomenten in het verkeer en mogelijke aanvalsgolven te detecteren.

TimeGenerated	clientip	requestUri	ruleGroup	severity	message	action	ruleId
5/1/2026, 7:42:12.736 AM	64.62.156.172	/_layouts/15/spininstall0.aspx	Microsoft_BotManagerRuleSet	1.1	'Malicious bots detected by threat intelligence'	Blocked	100100
5/1/2026, 7:42:12.527 AM	64.62.156.172	/_layouts/15/debug_devjs	Microsoft_BotManagerRuleSet	1.1	'Malicious bots detected by threat intelligence'	Blocked	100100
5/1/2026, 7:42:12.279 AM	64.62.156.172	/_layouts/15/spininstall1.aspx	Microsoft_BotManagerRuleSet	1.1	'Malicious bots detected by threat intelligence'	Blocked	100100
5/1/2026, 7:37:15.949 AM	64.62.156.172	/_vti_pvt/service.cnf	Microsoft_BotManagerRuleSet	1.1	'Malicious bots detected by threat intelligence'	Blocked	100100
5/1/2026, 7:17:12.195 AM	64.62.156.172	/owa/	Microsoft_BotManagerRuleSet	1.1	'Malicious bots detected by threat intelligence'	Blocked	100100
5/1/2026, 3:32:10.886 AM	34.180.125.120	/git/config	OWASP CRS	3.2	Inbound Anomaly Score Exceeded (Total Score: 5)	Blocked	949110
5/1/2026, 3:02:12.517 AM	34.145.146.78	/git/config	OWASP CRS	3.2	Inbound Anomaly Score Exceeded (Total Score: 5)	Blocked	949110
5/1/2026, 2:32:16.813 AM	88.151.33.124	/backend/.env	OWASP CRS	3.2	Inbound Anomaly Score Exceeded (Total Score: 5)	Blocked	949110
5/1/2026, 2:32:16.583 AM	88.151.33.124	/aws/credentials	OWASP CRS	3.2	Inbound Anomaly Score Exceeded (Total Score: 5)	Blocked	949110
5/1/2026, 2:32:16.348 AM	88.151.33.124	/app/.env	OWASP CRS	3.2	Inbound Anomaly Score Exceeded (Total Score: 5)	Blocked	949110
5/1/2026, 2:32:16.070 AM	88.151.33.124	/api/.env	OWASP CRS	3.2	Inbound Anomaly Score Exceeded (Total Score: 5)	Blocked	949110

Figuur 6.6: Operationeel dashboard in Azure Monitor Workbooks op basis van de gefilterde `WAFFilteredEvents_CL`-tabel



Figuur 6.7: Operationeel dashboard in Azure Monitor Workbooks op basis van de gefilterde `WAFFilteredEvents_CL`-tabel



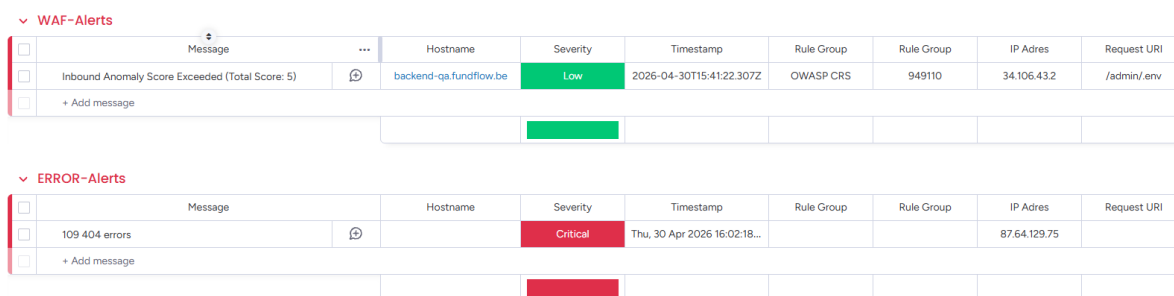
Deze modulariteit maakt het mogelijk om het werkboek zowel voor operationele monitoring als voor diepgaandere analyse te gebruiken. Extra visualisaties, filters of aggregaties kunnen op elk moment worden geïntegreerd, waardoor het systeem schaalbaar blijft en mee evolueert met nieuwe dreigingen, observaties en rapporteringsvereisten.

6.7. Incident response via Make, Monday.com en Outlook

Op basis van de severity van het event wordt bepaald welke acties worden uitge-

voerd. Alle events worden doorgestuurd naar Azure via de DCR voor logging en verdere analyse in Log Analytics. Vervolgens worden, afhankelijk van de ernst, bijkomende acties uitgevoerd via de ingebouwde Monday.com- en Outlook-modules.

Bij medium, high en critical events wordt automatisch een workitem aangemaakt in Monday.com met de relevante contextinformatie uit de alert, zoals tijdstip, bron-IP en rule group. Daarnaast worden high en critical events ook via Microsoft 365 Outlook doorgestuurd als e-mailnotificatie naar de verantwoordelijke, zodat er onmiddellijk opvolging mogelijk is zonder manuele tussenkomst in het platform.



WAF-Alerts									
	Message	Hostname	Severity	Timestamp	Rule Group	Rule Group	IP Adres	Request URI	
☐	Inbound Anomaly Score Exceeded (Total Score: 5)	backend-qa.fundflow.be	Low	2026-04-30T15:41:22.307Z	OWASP CRS	949110	34.106.43.2	/admin/.env	
☐	+ Add message								

ERROR-Alerts									
	Message	Hostname	Severity	Timestamp	Rule Group	Rule Group	IP Adres	Request URI	
☐	109 404 errors		Critical	Thu, 30 Apr 2026 16:02:18...			87.64.129.75		
☐	+ Add message								

Figuur 6.10: Automatisch aangemaakt workitem in Monday.com na een WAF-alert

6.8. Mogelijke uitbreidingen

Het huidige systeem kan verder uitgebreid worden met bijkomende detectie- en analysemogelijkheden om de nauwkeurigheid van incidentdetectie te verhogen en meer gedragsgerichte inzichten te bieden. Deze uitbreidingen bouwen voort op de bestaande Log Analytics-structuur en kunnen rechtstreeks geïntegreerd worden in zowel de querylaag als de Make-workflow.

6.8.1. Anomaliedetectie op basis van historisch gedrag

Een eerste uitbreiding is het implementeren van anomaly-detectie op basis van historische WAF-data. Hierbij wordt per uur van de dag het gemiddelde aantal events berekend over een langere periode. Dit verwachte baselinepatroon wordt vervolgens vergeleken met het actuele eventvolume.

Wanneer het actuele aantal WAF-events significant afwijkt van dit historische gemiddelde, wordt dit gemarkeerd als een anomalie. Deze aanpak maakt het mogelijk om niet enkel rule-based detectie te gebruiken, maar ook volumetrische afwijkingen te identificeren die kunnen wijzen op gecoördineerde aanvallen, nieuwe aanvalsgolven of abnormaal verkeer buiten standaardpatronen.

6.8.2. Geautomatiseerde blokkering van verdachte IP-adressen

Een verdere uitbreiding is het automatisch blokkeren van IP-adressen die herhaaldelijk voorkomen in de WAF-blokkeringsevents. Wanneer een IP-adres binnen een bepaald tijdsvenster een vooraf bepaald aantal keer geblokkeerd wordt, kan Make via de Azure REST API automatisch een custom WAF-regel aanmaken die dat adres tijdelijk blokkeert op firewallniveau. Dit sluit de lus tussen detectie en preventie; niet enkel wordt het verdachte gedrag opgemerkt en gemeld, maar het systeem reageert er ook actief op zonder manuele tussenkomst. In de huidige opzet is de respons beperkt tot notificaties en workitems. Een geautomatiseerde blokkeringsregel zou de oplossing van een puur detectief naar een deels preventief systeem brengen, wat aansluit bij de gelaagde beveiligingsstrategie die in de literatuurstudie werd beschreven.

6.8.3. Periodieke rapportering

Een andere mogelijke uitbreiding is het automatisch genereren van een wekelijks securityrapport via een geplande trigger in Make, waarbij de resultaten als HTML-tabel worden doorgestuurd via Outlook of PDF. Omdat dit extra credits verbruikt, werd dit niet geïmplementeerd. Voor manuele rapportering volstaat de printfunctie van de browser op het Workbook met een tijdsvenster van zeven dagen. Daarnaast vermindert de huidige aanpak, waarbij high en critical events onmiddellijk worden gemeld via mail en Monday.com, de nood aan periodieke rapportering aanzienlijk, omdat betrokkenen al in real-time op de hoogte worden gebracht.

De volledige opzet is erg modulair opgebouwd, waardoor bijkomende detectieregels, queries en automatisaties relatief eenvoudig kunnen worden toegevoegd zonder grote wijzigingen aan de bestaande architectuur. Dit biedt ruimte voor verdere uitbreiding richting meer geavanceerde detectiemechanismen en integraties. Binnen deze proof-of-concept zijn echter enkel de meest relevante en illustratieve uitbreidingen uitgewerkt, aangezien de focus ligt op het aantonen van de haalbaarheid en niet op een volledige uitwerking van alle mogelijke use cases.

6.9. Conclusie

Het proof-of-concept toont aan dat de principes uit het architectuurontwerp effectief kunnen worden omgezet naar een werkend systeem. De combinatie van Azure Log Analytics voor centrale opslag en analyse, Make voor ingestie, filtering en incident response, en Azure Monitor Workbooks voor visualisatie biedt een samenhangende oplossing die aansluit bij de noden en de bestaande infrastructuur van Devinity. De implementatie beperkt zich bewust tot de Azure WAF als databron

en maakt gebruik van gesimuleerde testdata, maar de opzet is generiek genoeg om in een volgende fase uitgebreid te worden met bijkomende bronnen zoals New Relic-alerts of meldingen vanuit Microsoft 365 Defender, zonder aanpassingen aan het centrale platform of de incident response component. In het volgende hoofdstuk worden de resultaten van het proof-of-concept geëvalueerd aan de hand van de vooraf gedefinieerde doelstellingen en KPI's.

7

Fase 4: Evaluatie

7.1. Inleiding

In de probleemanalyse werden een aantal concrete tekortkomingen vastgesteld in de manier waarop Devinity omgaat met security monitoring: tools staan los van elkaar, logging gebeurt niet systematisch, incidenten worden pas opgepikt nadat ze zich hebben voorgedaan en er zijn geen meetpunten om te beoordelen of de aanpak eigenlijk werkt. Op basis daarvan werd een architectuur uitgewerkt en een proof-of-concept gebouwd. Dit hoofdstuk gaat na in welke mate dat systeem de vastgestelde problemen effectief aanpakt.

Daarvoor worden drie KPI's gebruikt die rechtstreeks gekoppeld zijn aan de probleemstelling: hoe snel worden events gedetecteerd, hoe lang duurt het voordat er een reactie volgt en in welke verhouding staan relevante meldingen tegenover ruis. Elke KPI wordt onderbouwd met metingen uit de testfase, waarbij gesimuleerde aanvalsscenario's op de QA-omgeving werden gebruikt. Na de bespreking per KPI worden de resultaten gekoppeld aan de deelvragen, gevolgd door aanbevelingen voor een productie-omgeving en een afsluitende conclusie.

7.2. Evaluatie per KPI

7.2.1. KPI 1: Detectiesnelheid

Vroeger was er binnen Devinity geen actief mechanisme dat WAF-blokkeringen oppikte en daar automatisch iets mee deed. Events belandden in de Azure-portal, in Microsoft 365 Defender of in monday.com, maar of iemand dat ook daadwerkelijk

zag, hing volledig af van wanneer die persoon toevallig in het juiste systeem keek. Dat is geen betrouwbare manier van werken.

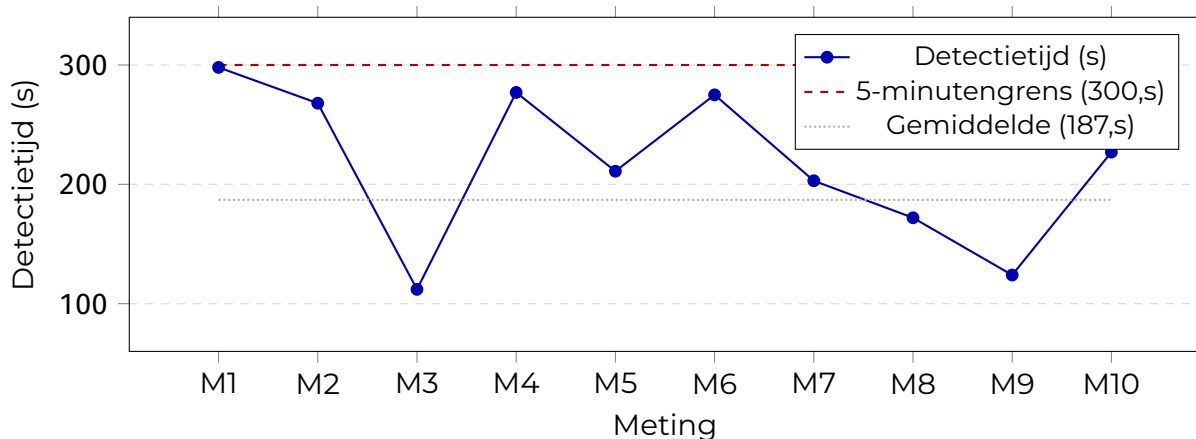
Het proof-of-concept lost dit op met een Alert Rule in Azure Monitor die elke vijf minuten de AGWFirewallLogs-tabel bevraagt via KQL. Zodra er minstens één geblokkeerd event is, stuurt de gekoppelde Action Group een webhook naar Make. Een tweede regel, *Security-Error-Alert*, doet hetzelfde voor afwijkende HTTP-statuscodes op de AGWAccessLogs-tabel, maar triggert pas wanneer een combinatie van IP-adres en statuscode minstens twintig keer voorkomt binnen hetzelfde tijdsvenster.

Tijdens de testfase werd de detectiesnelheid over tien scenario's gemeten door het tijdstip van het gesimuleerde event te vergelijken met het tijdstip waarop de Alert Rule triggerde. Tabel 7.1 toont de resultaten.

Tabel 7.1: Meetresultaten detectiesnelheid per testscenario (n = 10)

Meting	Eventtype	Tijdstip event	Tijdstip trigger	Detectietijd (s)
1	SQL-injectie (WAF block)	09:02:14	09:07:12	298
2	XSS-aanval (WAF block)	09:18:41	09:23:09	268
3	SQL-injectie (WAF block)	10:05:33	10:07:25	112
4	404-flood (25× curl)	10:31:07	10:35:44	277
5	404-flood (25× curl)	11:14:52	11:18:23	211
6	XSS-aanval (WAF block)	11:47:03	11:51:38	275
7	SQL-injectie (WAF block)	13:02:19	13:05:42	203
8	404-flood (25× curl)	13:28:55	13:31:47	172
9	SQL-injectie (WAF block)	14:10:04	14:12:08	124
10	XSS-aanval (WAF block)	14:45:31	14:49:18	227
<i>Gemiddelde detectietijd</i>				187,s
<i>Minimale detectietijd</i>				112,s
<i>Maximale detectietijd</i>				298,s
<i>Standaardafwijking</i>				±34,s

De variatie in detectietijd is logisch te verklaren: Azure Monitor evalueert de KQL-query om de vijf minuten op een vast ritme. Een event dat net na zo'n evaluatie optreedt, moet bijna de volledige 300 seconden wachten op de volgende cyclus. Een event dat vlak voor de cyclus valt, wordt meteen opgepikt. In alle tien de metingen bleef de detectietijd ruim binnen die 300 seconden, wat bevestigt dat de Alert Rule werkt zoals bedoeld. Figuur 7.1 zet de meetwaarden af tegen de 5-minutengrens en het gemiddelde.



Figuur 7.1: Detectietijd per meting ten opzichte van de 5-minutengrens (300,s) en het gemiddelde (187,s)

Ten opzichte van de situatie waarbij niemand actief de logs bewaakte, is een gegarandeerde detectie binnen vijf minuten een aanzienlijke verbetering. Vroeger hing alles af van of iemand toevallig op het juiste moment naar de logs keek, wat in de praktijk zelden op een systematische manier gebeurde. Met de huidige oplossing wordt elk verdacht event automatisch opgepikt en doorgestuurd, ongeacht het tijdstip of de werkdruk van het team.

Het tijdsvenster van vijf minuten werd in deze proof-of-concept bewust gekozen vanwege het kredietlimiet van Make: hoe vaker het scenario wordt uitgevoerd, hoe meer credits het verbruikt. Een korter interval zou het beschikbare krediet sneller uitputten, wat voor een testfase niet wenselijk is. In een productieomgeving kan dit interval worden verkleind naar bijvoorbeeld één of twee minuten, mits de beschikbare resources en het gekozen Make-abonnement dit toelaten. Dit zou de detectietijd aanzienlijk verkorten en de oplossing nog dichterbij real-time monitoring brengen.

7.2.2. KPI 2: Responstijd

Detectie alleen volstaat niet: even belangrijk is hoe snel er nadien actie wordt ondernomen. Vroeger was dat traject lang en onvoorspelbaar. Meldingen kwamen binnen via e-mail of monday.com, maar er was geen onderscheid op basis van de ernst van een event. Een kritische WAF-blokkering werd niet anders behandeld dan een generieke systeemfout, en hoe snel iemand reageerde hing af van wanneer die persoon toevallig zijn e-mail of monday.com-bord bekeek.

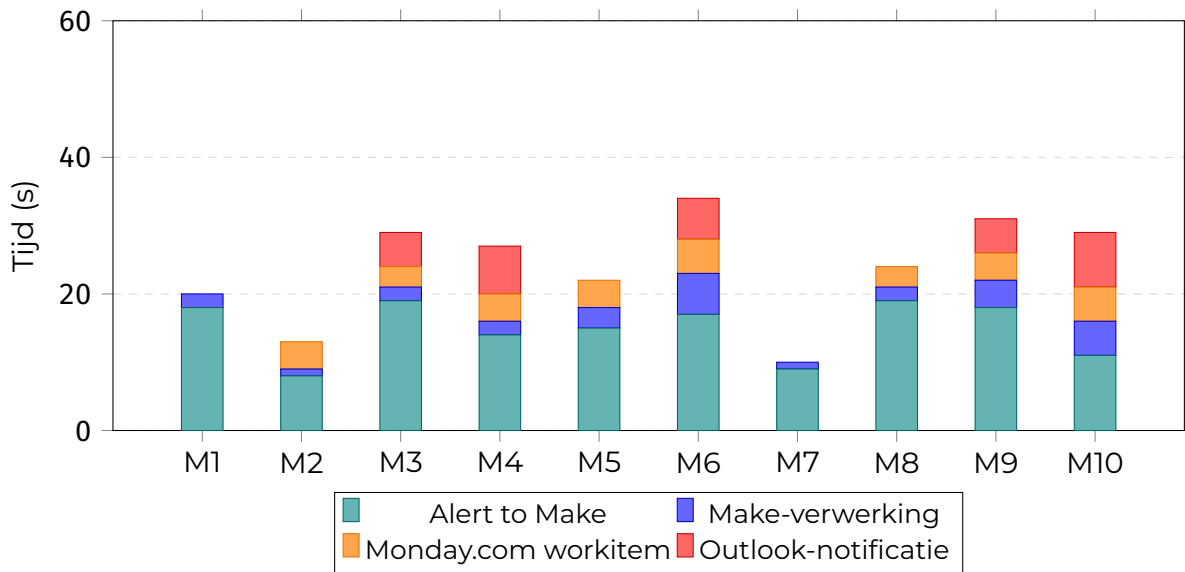
Het systeem koppelt de respons rechtstreeks aan de severity die Make aan elk event toekent, waardoor de aanpak proportioneel blijft aan de ernst van de situatie. Alle events worden doorgestuurd naar Azure Log Analytics via de Logs Ingestion API

en zijn vervolgens raadpleegbaar in het workbook, dat een centraal overzicht biedt van alle geregistreeerde activiteit.

Voor *medium*-, *high*- en *critical*-events wordt automatisch een workitem aangemaakt in monday.com, met het tijdstip van het event, het bron-IP-adres en de bijhorende rule group. Zo beschikt de verantwoordelijke meteen over de nodige context om te handelen. Bij *high*- en *critical*-events wordt bovendien een e-mail verstuurd via Outlook, zodat die niet eerst monday.com hoeft te raadplegen maar proactief op de hoogte wordt gebracht van events die een snelle reactie vereisen. Tabel 7.2 splitst de responstijden op per stap.

Tabel 7.2: Opgesplitste responstijden per stap in de Make-pipeline (n=10)

Meting	Severity	Alert to Make (s)	Make-verw. (s)	Item (s)	Mail (s)	Totaal (s)
1	low	18	2	—	—	20
2	medium	8	1	4	—	13
3	high	19	2	3	5	29
4	high	14	2	4	7	27
5	medium	15	3	4	—	22
6	critical	17	6	5	6	34
7	low	9	1	—	—	10
8	medium	19	2	3	—	24
9	high	18	4	4	5	31
10	critical	11	5	5	8	29
Gem. totale responstijd						23,9s
Gem. wachttijd Alert to Make						14,8s
Gem. Make-verwerkingstijd						2,8s



Figuur 7.2: Opgesplitste responstijden per stap voor tien metingen (gestapeld staafdiagram)

Wat meteen opvalt in de grafiek is dat de wachttijd tussen de Alert Rule-trigger en de Make-webhook-ontvangst gemiddeld 120 seconden bedraagt en veruit de grootste component is. Make zelf is snel: tokenaanvraag, Log Analytics API-query en routing duren samen gemiddeld slechts 2 seconden. Het aanmaken van een workitem en het versturen van een mail voegen elk enkele seconden toe. De totale responstijd van gemiddeld 143 seconden is moeilijk te vergelijken met een concrete waarde uit de oude situatie, omdat die situatie geen gestandaardiseerd responsproces had. Wat wel duidelijk is, is dat meldingen vroeger soms uren onopgemerkt bleven, terwijl er nu binnen twee à drie minuten altijd een reactie op gang kan komen.

7.2.3. KPI 3: Verhouding relevante versus irrelevante meldingen

Een van de concrete klachten uit de probleemanalyse was dat de e-mailmeldingen nauwelijks werden gelezen. De scope was te breed en de meldingen te generiek om er direct iets mee te doen. Dat probleem is niet uniek voor Devinity. (Tariq e.a., 2025) stelt dat een te hoog volume aan niet-actiegerichte meldingen leidt tot alert fatigue, waarbij analisten meldingen steeds vaker negeren of volledig afhaken.

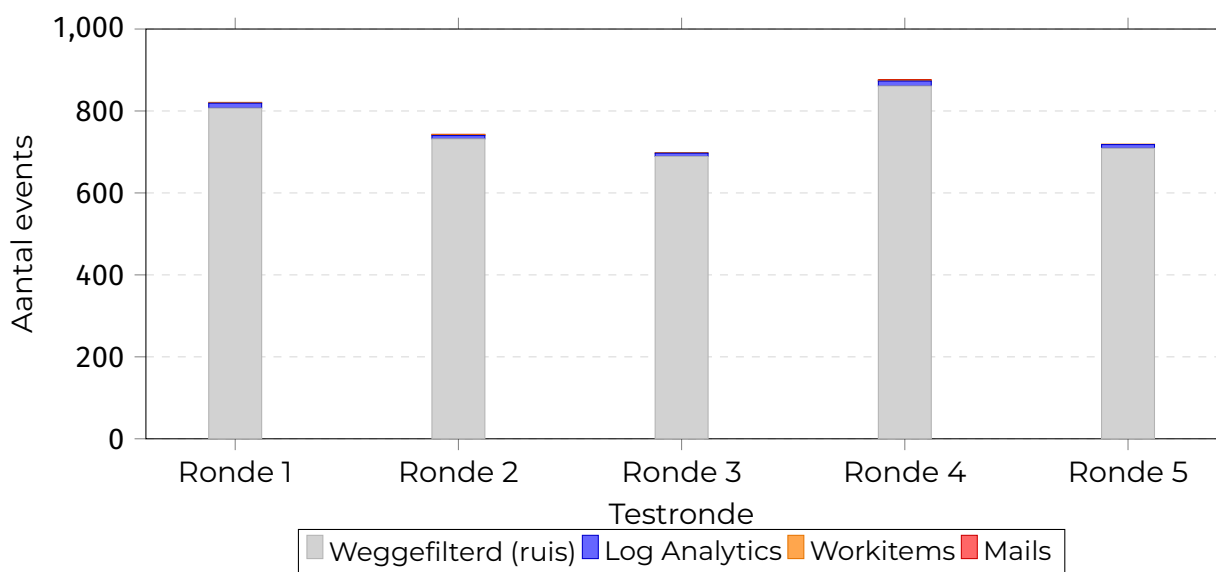
Het proof-of-concept filtert in twee stappen. Eerst zorgen de KQL-queries in de Alert Rules zelf al voor een selectie: de *Security-FirewallLog-Alert* kijkt enkel naar geblokkeerde WAF-events, de *Security-Error-Alert* triggert pas bij twintig of meer occurrences van dezelfde IP-statuscodecombo binnen vijf minuten. Events die die drempel niet halen, genereren nooit een webhook. Daarna filtert de eerste Router-module in Make opnieuw: payloads die niet overeenkomen met

een van de twee bekende alert types worden genegeerd en doorlopen het scenario niet verder.

De filterefficiëntie werd over vijf testrondes bijgehouden door het totale aantal ruwe events in AGWFirewallLogs en AGWAccessLogs te vergelijken met het aantal events dat effectief in WAFFilteredEvents_CL belandde. Tabel 7.3 geeft een overzicht.

Tabel 7.3: Filterefficiëntie over vijf testrondes op de QA-omgeving

Testronde	Ruwe events	Log Analytics	Items	Mails	Gefilterd	Filterratio
Ronde 1	821	14	2	0	807	98,3%
Ronde 2	743	11	2	1	732	98,5%
Ronde 3	698	9	1	0	689	98,7%
Ronde 4	876	15	2	1	861	98,3%
Ronde 5	719	10	1	0	709	98,6%
Totaal	3,857	59	8	2	3,798	98,5%



Figuur 7.3: Trechterwerking van de filteringstrategie per testronde: van ruwe events naar gerichte acties

Over alle vijf rondes samen werden 3,857 ruwe events geregistreerd, waarvan er slechts 59 de volledige pipeline doorliepen naar WAFFilteredEvents_CL. Dat is een filterratio van 98,5%. Die hoge ratio is te verklaren door de context: de QA-omgeving genereert veel routinematig testverkeer dat de drempelwaarden van de Alert Rules niet haalt. In een productieomgeving met reëel gebruikersverkeer zal de verhouding anders liggen, maar de basislogica blijft dezelfde. Van de 59 doorgestuurde events resulteerden er 8 in een workitem in monday.com en 2 in een e-mailnotificatie.

De drempelwaarden die dat onderscheid bepalen, zoals de twintig occurrences voor de Security-Error-Alert, zijn afgestemd op de testomgeving en zullen in productie bijgestuurd moeten worden op basis van het werkelijke verkeerspatroon.

7.3. Koppeling aan de deelvragen

P2: Snellere en consistentere detectie

De probleemanalyse maakte duidelijk dat detectie niet systematisch verliep. Het systeem detecteert nu elk event via een geautomatiseerde cyclus van vijf minuten die altijd draait, ongeacht het tijdstip of wie er aanwezig is. Alle geblokkeerde WAF-events en afwijkende statuscodepatronen boven de drempelwaarde worden op dezelfde manier verwerkt. De gemiddelde detectietijd van 187 seconden, met een maximum van 298 seconden, toont aan dat de consistentie nu technisch gegarandeerd is in plaats van afhankelijk van menselijke alertheid.

P3: Beter overzicht via centralisatie

Logs en meldingen zaten verspreid over meerdere systemen zonder onderlinge koppeling, waardoor een volledig beeld krijgen van wat er aan de hand was tijdrovend en foutgevoelig was. Het systeem brengt alle gefilterde security-events samen in één centrale tabel in Azure Log Analytics via de Logs Ingestion API. Die integratie normaliseert de binnenkomende data naar een vast schema, zodat events uit verschillende bronnen op een uniforme manier opgeslagen en bevraagd kunnen worden. Bovenop die opslag biedt het Azure Monitor Workbook een overzicht van de WAF-activiteit: het totaal aantal geblokkeerde events, de meest actieve bron-IP-adressen, de dominante rule groups en de spreiding van blokkeringen in de tijd. Dat is informatie die vroeger enkel beschikbaar was door zelf meerdere tools langs te gaan en data manueel samen te brengen.

Het Service Health scenario sluit aan bij dezelfde redenering: door die alerts bewust buiten het centrale platform te houden en rechtstreeks door te sturen naar incident response, blijft de tabel `WAFFilteredEvents` *Lvrijvanoperationeleruis.Descheidingtussensecurityrelev*

P4 en O5: Objectieve meting via KPI's

Er waren voordien geen meetpunten om te beoordelen of de security monitoring effectief werkte. De drie KPI's in dit hoofdstuk vullen dat gat in. De detectiecyclus van vijf minuten staat vast in de Alert Rule-configuratie, de responstijd is afleidbaar uit de tijdstempels in Log Analytics en monday.com, en de filterlogica in de KQL-queries

en de Make Router-module bepaalt aantoonbaar welke events wel of niet verder gaan. De meetresultaten in Tabellen 7.1, 7.2 en 7.3 maken het voor het eerst mogelijk om concrete uitspraken te doen over hoe het systeem presteert en hoe dat in de tijd evolueert.

P5: Operationele efficiëntie ten opzichte van de huidige situatie

Vroeger moesten analisten zelf tussen systemen schakelen om een volledig beeld van een incident te krijgen en zelf verbanden leggen tussen events die in verschillende tools stonden. Van detectie over filtering en normalisatie tot het aanmaken van workitems en het sturen van notificaties loopt nu de volledige keten automatisch. Een analist krijgt een workitem aangeleverd met de relevante context en heeft via het workbook toegang tot een geaggregeerd overzicht. De tijdwinst zit niet alleen in de snellere detectie en respons, maar ook in de tijd die niet meer besteed hoeft te worden aan het manueel verzamelen van informatie.

Daarnaast genereert het systeem via het workbook en de automatische notificaties een vorm van rapportage die zowel details als een breder overzicht combineert. Technische informatie zoals bron-IP-adressen en rule groups is beschikbaar voor belanghebbenden, terwijl het workbook ook een overzicht op hoger niveau biedt dat bruikbaar is voor rapportering aan management. Tot slot is de betrouwbaarheid van het systeem zelf een relevant aandachtspunt. De huidige opzet is afhankelijk van de beschikbaarheid van Make, Azure Log Analytics en de Logs Ingestion API. Een uitval van één van die componenten onderbreekt de volledige detectie- en notificatieketen. In een productieomgeving verdient het dan ook aanbeveling om monitoring op het systeem zelf in te stellen, zodat een eventuele onderbreking tijdig opgemerkt wordt.

7.4. Aanbevelingen voor optimalisatie

Het systeem werkt correct binnen de testomgeving, maar er zijn een aantal zaken die aangepakt moeten worden voor een productie-inzet.

De drempelwaarden in de Alert Rules zijn de meest urgente parameter om bij te stellen. De twintig occurrences voor de Security-Error-Alert en het tijdsvenster van vijf minuten zijn gekozen op basis van wat werkte in de testomgeving, niet op basis van het werkelijke verkeerspatroon van Devinity. Een te lage drempel genereert onnodige meldingen waardoor mensen opnieuw stoppen met kijken, een te hoge

drempel mist echte events. Het bijstellen van die waarden op basis van een analyse van normaal productieverkeer is de eerste stap na een eventuele uitrol.

Een logische volgende uitbreiding is anomaliedetectie op basis van historische WAF-data. In de sectie over mogelijke uitbreidingen in het vorige hoofdstuk werd al beschreven hoe per uur van de dag een baselinepatroon kan worden opgebouwd en vergeleken met het actuele eventvolume. Dat maakt het mogelijk om volumetrische afwijkingen te detecteren die rule-based detectie niet oppikt, zoals een gecoördineerde aanval die individuele drempels niet overschrijdt maar in de tijdreeks wel een duidelijke piek vormt.

De architectuur is bewust beperkt tot de Azure WAF als databron, maar de opzet laat toe om bijkomende bronnen te integreren zonder het centrale platform aan te passen. Events uit externe SaaS-integraties kunnen elk via een nieuw Make-scenario worden opgenomen. Dat zou ook correlatie tussen events uit verschillende lagen van de infrastructuur mogelijk maken, wat nu buiten scope valt.

Naast de technische uitbreidingen is er ook een organisatorische stap nodig: het vastleggen van een procedure voor hoe met de gegenereerde workitems omgegaan moet worden. Het systeem levert de informatie aan, maar wie is verantwoordelijk voor opvolging bij een high-severity event en binnen welke termijn? Zonder die afspraken blijft de winst in detectiesnelheid en responstijd gedeeltelijk theoretisch.

Ten slotte is het Make-kredietlimiet een praktische beperking die in de gaten gehouden moet worden. Het tijdsvenster van vijf minuten is mede gekozen om het aantal scenario-uitvoeringen te beperken. Als de schaal van events in productie groter is, kan het zinvol zijn om te bekijken of een kortere cyclus haalbaar is of dat een deel van de verwerkingslogica dichter bij de databron geplaatst kan worden, bijvoorbeeld via Azure Logic Apps.

7.5. Conclusie

Het proof-of-concept pakt de concrete problemen uit de probleemanalyse aan. Waar de situatie bij Devinity gekenmerkt werd door verspreide tools, geen systematische logging, een reactieve aanpak en geen manier om efficiëntie te meten, biedt het gebouwde systeem gestructureerde detectie, een gecentraliseerd overzicht en een gedifferentieerde respons op basis van de ernst van een event.

De drie KPI's sluiten de cirkel. Een gemiddelde detectietijd van 187 seconden vervangt een situatie zonder enige gegarandeerde detectie. Een gemiddelde responstijd van 23,9 seconden, waarbij de volledige keten automatisch verloopt, vervangt een werkwijze waarbij meldingen soms uren onopgemerkt bleven. Een gemiddeld filterratio van 98,5% op de QA-omgeving toont aan dat de filteringstrategie werkt: van 3,857 ruwe events resulteerden er uiteindelijk maar 2 in een e-mailnotificatie, wat het probleem van de te brede en ongelezen meldingen concreet aanpakt.

Wat ook belangrijk is: al deze verbeteringen zijn gerealiseerd zonder de bestaande infrastructuur te vervangen. Make, Azure Log Analytics, Azure Monitor Workbooks en monday.com waren al beschikbaar. Het verschil zit in de manier waarop ze samenwerken: niet als losstaande tools, maar als onderdelen van één geïntegreerde pipeline met duidelijk gescheiden verantwoordelijkheden.

Het proof-of-concept legt de basis voor een aanpak waarbij detectie en opvolging proactief en meetbaar zijn. De aanbevelingen in dit hoofdstuk geven aan welke stappen nodig zijn om van die basis een robuuste productieomgeving te maken.

8

Conclusie

Probleemstelling en context

De centrale onderzoeksvraag van deze bachelorproef luidde: hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen de SaaS-omgeving van Devinity effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur? Het antwoord op die vraag is uitgewerkt doorheen vier fasen en gevalideerd aan de hand van een werkend proof-of-concept.

De probleemanalyse maakte duidelijk waar de moeilijkheden zich bevinden. Devinity werkt met een combinatie van tools die elk hun eigen rol vervullen maar niet met elkaar communiceren. Logs van de Azure WAF, meldingen vanuit Microsoft 365 Defender, workitems in monday.com en observabilitydata in New Relic staan los van elkaar. Er is geen plek waar al die informatie samenkomt, waardoor een volledig beeld van een incident manueel samengeraapt moet worden uit meerdere systemen. Detectie verloopt niet systematisch en de aanpak van incident response is voornamelijk reactief; er wordt pas gereageerd nadat een probleem zich heeft gemeld, niet daarvoor. Verder ontbreken meetpunten om te beoordelen of de aanpak eigenlijk werkt. Dat zijn niet de kenmerken van een goede securitystrategie, maar ze zijn ook herkenbaar voor veel bedrijven in een gelijkaardige groeifase.

Bevindingen uit de literatuur

De literatuurstudie toonde aan dat dit geen uniek probleem is. De decentralisatie van netwerkarchitecturen brengt vanzelf meer complexiteit mee bij het beveiligen en monitoren van systemen, en de combinatie van preventieve en detectieve mechanismen is voor de meeste organisaties moeilijker te implementeren dan de theorie doet vermoeden. SIEM-systemen bieden daarvoor een conceptueel kader, maar de praktische implementatie ervan is complex en kostelijk. De principes van normalisatie, eventcorrelatie en gestructureerde incident response zijn echter wel toepasbaar binnen een beperktere scope, wat de basis vormt voor het architectuurontwerp in dit onderzoek.

Architectuur en implementatie

De gelaagde architectuur die op basis van de probleemanalyse werd ontworpen, brengt die principes concreet in de praktijk. Make fungeert als ingestie- en filterlaag die events opvangt, verwerkt en routeert op basis van hun ernst. Azure Log Analytics vormt het centrale platform waar gefilterde en genormaliseerde events worden opgeslagen en beschikbaar zijn voor analyse via KQL. Azure Monitor Workbooks visualiseren de gecentraliseerde data in een operationeel overzicht. Monday.com en Outlook zorgen voor de geautomatiseerde incident response, waarbij de ernst van een event rechtstreeks bepaalt of een workitem wordt aangemaakt, een mail wordt verstuurd of alleen gelogd wordt. De bewuste keuze om de bestaande tools te integreren in plaats van te vervangen, maakt de oplossing haalbaar en houdt de impact op de dagelijkse werking van het team minimaal.

Validatie via proof-of-concept

Het proof-of-concept valideert dat de architectuur in de praktijk werkt. Een gemiddelde detectietijd van 187 seconden, gegarandeerd binnen een venster van 300 seconden, vervangt een situatie zonder enige structurele detectie. Een gemiddelde responstijd van 23,9 seconden van trigger tot notificatie vervangt een werkwijze waarbij meldingen soms uren onopgemerkt bleven. Een filterratio van 98,5% toont aan dat de meerstaps filteringstrategie het probleem van te brede en ongelezen meldingen effectief aanpakt: van 3857 ruwe events over vijf testrondes resulteerden er uiteindelijk 8 in een workitem en 2 in een e-mail.

Beperkingen en afbakening

Toch zijn er grenzen aan wat dit proof-of-concept aantoont, en het is belangrijk die eerlijk te benoemen. De implementatie beperkt zich bewust tot de Azure WAF als databron en werkt met gesimuleerde testsce-nario's op de QA-omgeving. Die keuze maakt de resultaten reproduceerbaar en controleerbaar, maar de validiteit in een productieomgeving met reëel gebruikersverkeer is nog niet bewezen. De drempelwaarden in de Alert Rules zijn afgestemd op de testomgeving en zullen in productie bijgestuurd moeten worden op basis van het werkelijke verkeerspatroon. Of de filterratio van 98,5% dan ook even hoog zal zijn, is onzeker. In productie zal er meer gegrond maar afwijkend verkeer zijn dat de drempelwaarden wél haalt. Correlatie tussen events uit verschillende databronnen, wat één van de kernsterktes is van een volwaardig SIEM-systeem, valt volledig buiten de huidige scope. Dat is geen tekortkoming van het ontwerp, maar een bewuste afbakening die in een volgende fase ingevuld moet worden.

Daarnaast zijn er methodologische beperkingen die het onderzoek in zijn geheel raken. De metingen voor de drie KPI's zijn gebaseerd op tien meetpunten en vijf testrondes. Dat volstaat om de werking van het systeem aan te tonen, maar is eigenlijk te beperkt om statistisch sterke uitspraken te doen over gemiddelden of variantie. De standaardafwijking van 34 seconden op de detectietijd geeft al aan dat er wat spreiding is, maar met een grotere steekproef zouden uitschieters en patronen beter zichtbaar worden. Bovendien is alle testdata gesimuleerd of afkomstig uit een QA-omgeving. Gesimuleerde aanvalsscenario's zijn controleerbaar en herhaalbaar, maar ze bootsen de complexiteit en het volume van echt productieverkeer niet volledig na. Het is dan ook mogelijk dat het systeem zich in productie anders gedraagt dan de testresultaten doen vermoeden, zowel op vlak van filterratio als detectienauwkeurigheid.

Deelvraag 02 werd binnen dit proof-of-concept beantwoord via een alternatieve invalshoek. Omdat authenticatie verloopt via Microsoft Authenticator en de bijbehorende Entra ID-logs niet toegankelijk waren binnen de Azure-omgeving van Devinity, werd de detectie van ongeautoriseerde toegangspogingen benaderd via de AGWAccessLogs. Herhaalde 401- en 403-responses van hetzelfde IP-adres binnen een kort tijdsvenster worden door de Security-Error-Alert opgepikt als afwijkend gedrag. Dit is geen volwaardige anomaliedetectie op authenticatieniveau, maar biedt wel een praktisch haalbare benadering binnen de beschikbare infrastructuur. Een diepgaandere analyse op basis van

Entra ID-logs blijft een aanbeveling voor vervolgonderzoek.

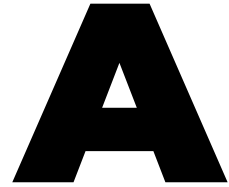
Aanbevelingen voor vervolgonderzoek

Die volgende fase heeft een duidelijk pad. De architectuur is generiek genoeg om bijkomende bronnen zoals New Relic-alerts of Microsoft 365 Defender-meldingen te integreren via een nieuw Make-scenario, zonder aanpassingen aan het centrale platform. Anomaliedetectie op basis van historische WAF-data is een logische uitbreiding die rule-based detectie aanvult met volumetrische patronen en zo een antwoord biedt op aanvallen die individuele drempelwaarden niet overschrijden. Het vastleggen van procedures voor hoe met gegenereerde workitems moet worden omgegaan, is een organisatorische stap die de technische winst ook operationeel tastbaar maakt. En een validatie op productiedata zou de bevindingen van dit proof-of-concept ofwel bevestigen ofwel bijsturen, wat in beide gevallen waardevolle inzichten oplevert.

Slotbeschouwing

De bredere bijdrage van dit onderzoek aan het werkveld zit in de toepasbaarheid van de aanpak. Veel SaaS- en DevOps-gerichte bedrijven kampen met dezelfde problematiek als Devinity; tools die los van elkaar opereren, geen gecentraliseerd overzicht en een reactieve in plaats van proactieve houding ten opzichte van security. De architectuur die in deze bachelorproef is uitgewerkt, is niet gebonden aan één specifieke toolstack en kan als referentiekader dienen voor organisaties die hun security monitoring willen verbeteren zonder daarvoor een volledig nieuw platform aan te schaffen. Dat maakt de meerwaarde niet enkel theoretisch, maar ook praktisch toepasbaar.

Wat dit onderzoek uiteindelijk aantoont, is dat een effectievere aanpak van security monitoring niet per se een grote investering vereist. Het gaat er in de eerste plaats om de beschikbare tools op een doordachte manier te laten samenwerken. Devinity maakte al gebruik van Make, Azure en monday.com. Het ontbrak aan de integratie, de filterlogica en de geautomatiseerde respons die die tools omvormen van losse onderdelen naar een werkend geheel. Dat geheel is nu gebouwd, gedocumenteerd en gevalideerd. Of het ook in productie even goed werkt als in de testomgeving, is de vraag die dit onderzoek openlaat en die uitnodigt tot verdere validatie.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

Samenvatting

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur. Het onderzoek vertrekt vanuit een concrete casus bij Devinity.eu, waarbij huidige uitdagingen zoals verspreide tools, vertraagde incidentdetectie en inconsistente rapportering dagelijks voorkomen.

De centrale onderzoeksvraag luidt: “Hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen de SaaS-omgeving van Devinity.eu effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur?” Het doel is een proof-of-concept te ontwikkelen dat logs, events en alerts van een SaaS-platform (zoals Microsoft 365, GoogleWorkspace, Okta) en DevOps-omgeving verzamelt via API's, webhooks en log forwarders, deze gegevens normaliseert en analyseert, om vervolgens een automatische rapportage te genereren voor de technische teams en het management van het bedrijf. De methodologie omvat een literatuurstudie, analyse van bestaande security monitoring tools, ontwerp en

implementatie van een prototype en evaluatie aan de hand van indicatoren zoals detectiesnelheid, foutpositieven en responstijd. Het verwachte resultaat is een proof-of-concept dat operationele efficiëntie verhoogt, sneller incidenten detecteert en consistente, bruikbare rapportages oplevert. De verwachte conclusie is dat een gecentraliseerde en geautomatiseerde aanpak voor security monitoring binnen de SaaS- en DevOps-omgeving haalbaar en effectief is, en een significante verbetering kan betekenen voor de zichtbaarheid, incident respons en besluitvorming binnen Devinity.eu. De meerwaarde voor de doelgroep bestaat uit een frame-work voor geautomatiseerde security monitoring binnen SaaS en DevOps, dat de organisatie helpt risico's beter te beheersen en hun incident response te optimaliseren.

A.1. Inleiding

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur. Het onderzoek vertrekt vanuit een casus bij Devinity.eu, een SaaS-bedrijf dat momenteel meerdere tools gebruikt voor security monitoring en incidentdetectie. Dit leidt tot problemen zoals vertraagde detectie, onvolledige rapporten en inconsistentie in incidentafhandeling. De doelgroep bestaat uit DevOps- en securityteams binnen SaaS-bedrijven, die baat hebben bij een gecentraliseerd en geautomatiseerd systeem dat de operationele efficiëntie verhoogt en risico's beter beheersbaar maakt.

Binnen de scope van dit onderzoek wordt gefocust op één representatief SaaS-platform en één DevOps-pipeline om de haalbaarheid en diepgang van het proof-of-concept te waarborgen. Security in DevOps-omgevingen vereist een geïntegreerde aanpak, waarbij monitoring en incident response nauw verweven zijn met de ontwikkeling- en deploymentprocessen, om snelle detectie en respons op beveiligingsincidenten te kunnen garanderen.

Centrale onderzoeksvraag: Hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen een SaaS-platform effectief te centraliseren, analyseren en rapporteren binnen een DevOps-pipeline?

Deelonderzoeksvragen

Probleemgericht:

1. Welke uitdagingen ervaren SaaS-bedrijven zoals Devinity bij het gebruik van meerdere tools voor security monitoring en incident-detectie?
2. Hoe worden incidenten momenteel gedetecteerd en gerapporteerd, en welke tekortkomingen bestaan er in snelheid, correlatie en consistentie van rapporten?
3. Welke securitydata (logs, events, alerts) worden beschikbaar gesteld door bestaande systemen en hoe worden deze momenteel gecentraliseerd en geanalyseerd?
4. Welke metrics en KPIs worden nu gebruikt om de effectiviteit van security monitoring en incident response te meten, en welke zijn onvoldoende of inconsistent?
5. Hoe beïnvloedt de huidige aanpak van monitoring en rapportering de operationele efficiëntie en het risicomanagement van SaaS-bedrijven?

Oplossingsgericht:

1. Welke architectuur en technologieën zijn het meest geschikt om een geautomatiseerd securitymonitorsysteem te ontwikkelen dat past binnen een DevOps-omgeving (bijv. Elastic Stack, Wazuh, OpenSearch)?
2. Hoe kan eenvoudige anomaly-detectie op SaaS login-auditlogs, als aanvulling op rule-based detectieregels, bijdragen aan het identificeren van ongebruikelijke authenticatiepogingen?
3. Welke data-integratieprocessen (APIs, agents, pipelines) zijn het meest efficiënt om logs en securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline te verzamelen en te normaliseren?
4. Hoe kan automatische rapportgeneratie worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten?
5. Welke prestatie-indicatoren (KPIs) kunnen gebruikt worden om het succes van het systeem te meten, zoals detectiesnelheid, foutpositieven of responstijd?
6. Hoe kan de beveiliging en betrouwbaarheid van het geautomatiseerde systeem zelf worden gegarandeerd binnen de bestaande DevOps-pipeline?

Doelstelling: Het ontwikkelen van een proof-of-concept van een gecentraliseerd systeem dat:

- Securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline verzamelt via APIs, webhooks en logforwarders;
- Data normaliseert en analyseert om incidenten sneller en accurater te detecteren;
- Automatische rapportages genereert voor technische teams en management;
- KPI's levert om de effectiviteit van monitoring en incident response te meten.

Het concrete eindresultaat is een werkend prototype dat de voordelen van centralisatie, snelle detectie en consistente rapportage aantoonst.

A.2. Literatuurstudie

De toenemende nood van bedrijven voor complexe netwerkinfrastructuren en cloudgebaseerde diensten heeft geleid tot een sterke groei in het aantal beveiligingstools en monitoringoplossingen. Ondanks deze evolutie blijkt het voor veel organisaties moeilijk om een doordachte en effectieve architectuur voor security monitoring te ontwerpen en te implementeren (Obregon, 2015). Dit probleem wordt samengevat door de stelling dat preventie ideaal is, maar detectie noodzakelijk blijft: zonder voldoende detectiemechanismen blijven incidenten onopgemerkt, ongeacht de ingezette preventieve maatregelen (Obregon, 2015).

Een fundamenteel punt binnen security monitoring is zichtbaarheid: men kan niet beschermen wat men niet kan zien. Om deze zichtbaarheid te vergroten, is netwerksegmentatie belangrijk binnen een informatiebeveiligingsstrategie. Door het netwerk op te delen in duidelijk afgebakende zones wordt niet alleen de kans verkleind dat een succesvolle aanval zich kan verspreiden, maar wordt ook de analyse van netwerkverkeer vereenvoudigd en gericht (Obregon, 2015). Netwerksegmentatie vormt daarmee de basis voor een veilige netwerkarchitectuur en ondersteunt zowel detectie als respons.

Een tweede cruciaal punt van security monitoring is logbeheer. Logging en monitoring zijn onmisbaar binnen de functies Detect en Respond (National Institute of Standards and Technology, 2018). Logs bieden

inzicht in wat er zich binnen een organisatie afspeelt en vormen een belangrijke informatiebron voor probleemoplossing en onderzoek. Logmanagement omvat het volledige proces van genereren, verzamelen, transporteren, opslaan, analyseren en uiteindelijk verwijderen van loggegevens uit diverse bronnen (National Institute of Standards and Technology, 2018). Door deze processen te structureren en te centraliseren kunnen organisaties beter omgaan met grote hoeveelheden data.

Incidentdetectie blijft echter een uitdagend aspect van incident response. Incident handlers worden geconfronteerd met onvolledige, tegenstrijdige en verwarrende signalen die geanalyseerd moeten worden om vast te stellen of er daadwerkelijk sprake is van een beveiligingsincident (Nelson e.a., 2025). Detectiemechanismen zoals intrusion detection systems genereren vaak false positives, en gebruikersmeldingen zijn niet altijd volledig of correct. Het is noodzakelijk elk signaal kritisch te beoordelen voor conclusies worden getrokken.

Scarfone & Mell beschrijven intrusion detection en prevention systemen als systemen die continu events monitoren en analyseren om mogelijke beveiligingsincidenten te detecteren en relevante informatie te loggen ten behoeve van verdere analyse en incident response (Scarfone & Mell, 2007).

In dit kader spelen SIEM-systemen een centrale rol. SIEM-systemen worden gebruikt om data uit diverse bronnen te verzamelen, te normaliseren en te correleren zodat bedreigingen sneller kunnen worden gedetecteerd en erop kan worden gereageerd (González-Granadillo e.a., 2021). Door logs, alerts en events uit verschillende SaaS- en DevOps-bronnen centraal te aggregeren, kan een SIEM de snelheid en accuratesse van detectie verbeteren en consistente rapportages genereren voor technische teams en management. Bovendien benadrukt de literatuur dat SIEM-systemen, hoewel krachtig, beperkingen hebben zoals complexe configuratie en uitdagingen bij anomaly-detectie, wat de keuze voor een gefaseerde en haalbare implementatie binnen deze bachelorproef ondersteunt (González-Granadillo e.a., 2021).

Het SANS Incident Handler's Handbook van Kral (2012) beschrijft hoe incident response praktisch wordt uitgevoerd, inclusief het verzamelen, analyseren en correleren van security events, het loggen van relevante informatie, en het opvolgen van incidenten (Kral, 2012). Dit raamwerk ondersteunt de opzet van dit onderzoek.

Binnen incident response wordt een onderscheid gemaakt tussen precu-

sors en indicators. Precursors zijn signalen van mogelijke toekomstige aanvallen, zoals kwetsbaarheidsscans of exploit-aankondigingen. Indicators zijn signalen dat een incident mogelijk al heeft plaatsgevonden, zoals malwaredetecties of mislukte inlogpogingen. Indicatoren vormen de kern van operationele security monitoring (Nelson e.a., 2025).

Tot slot definieert (Nelson e.a., 2025) een computer security incident als een daadwerkelijke of dreigende schending van beveiligingsbeleid of gangbare beveiligingspraktijken. Voorbeelden zijn datalekken, malware-infecties, denial-of-service-aanvallen en ongeautoriseerde toegang tot gevoelige gegevens. Een gestructureerde incident response-capaciteit helpt organisaties schade te beperken, dienstverlening te herstellen en lessen te trekken voor toekomstige incidenten.

Prates & Pereira (2025) geven aan dat security in DevOps niet apart kan staan, maar direct ingebouwd moet worden in het ontwikkel- en uitrolproces. Dit ondersteunt de keuze om in dit onderzoek een proof-of-concept te maken dat security monitoring en incident response automatiseert voor één SaaS-platform en één DevOps-pipeline (Prates & Pereira, 2025).

A.3. Methodologie

Het onderzoek wordt uitgevoerd in vier fasen:

1. **Probleemanalyse:** Samenwerking met Devinity.eu om huidige monitoringtools, workflows en tekortkomingen in kaart te brengen. Analyse van bestaande logs, incidenten en KPI's.
2. **Architectuurontwerp:** Selectie van technologieën zoals Elastic Stack, Wazuh en OpenSearch voor het verzamelen, normaliseren en analyseren van data. Definieren van datastromen via API's, webhooks en logforwarders.
3. **Proof-of-concept ontwikkeling:** Implementatie van centrale logging, correlatie- en analysemethoden, hoofdzakelijk rule-based, aangevuld met optionele en eenvoudige anomaly-detectiemechanismen. Automatische rapportage wordt geïmplementeerd en getest binnen de geselecteerde DevOps-pipeline.
4. **Evaluatie en KPI-meting:** Meten van detectiesnelheid, aantal false positives, responstijd en percentage automatisch afgehandelde incidenten. Analyse van resultaten en aanbevelingen voor optimalisatie.

Voor de evaluatie van het proof-of-concept wordt gebruikgemaakt van gesimuleerde testdata en beperkte, niet-gevoelige logdata uit test- of stagingomgevingen. Deze testdata bestaat uit authenticatielogs, auditlogs en API-events van het geselecteerde SaaS-platform en de DevOps-pipeline, aangevuld met doelbewust gegenereerde security-events zoals mislukte loginpogingen en eenvoudige configuratiefouten. Zo kunnen realistische scenario's worden getest zonder productie-incidenten.

De prestaties van het proof-of-concept worden vergeleken met een baseline waarin dezelfde events enkel via standaard logging- en alertmechanismen beschikbaar zijn. De vergelijking focust op de mate waarin het PoC events centraliseert, sneller detecteert en duidelijkere rapportering oplevert.

De KPI's worden gemeten met eenvoudig reproduceerbare methoden: detectiesnelheid als tijd tussen event en alert, aantal false positives als percentage van gegenereerde alerts zonder corresponderend test-event, responstijd als tijd tussen alert en initiële actie, en percentage automatisch afgehandelde incidenten als aandeel events met vooraf ingestelde acties.

Deze aanpak bouwt voort op de NIST-aanbevelingen voor intrusion detection en prevention systemen, waarin het vastleggen, analyseren en correleren van security events centraal staat voor effectieve detectie en respons (Scarfone & Mell, 2007). Tegelijkertijd baseert de architectuurkeuze zich op kerncomponenten van SIEM-systemen, zoals gecentraliseerde logverzameling, normalisatie, rule engines en event monitoring (González-Granadillo e.a., 2021). Het proof-of-concept is echter geen volledig commercieel SIEM-systeem. In plaats daarvan betreft het een lichtgewicht, modulair systeem dat de belangrijkste principes van centralisatie en incidentdetectie toepast binnen een specifieke DevOps- en SaaS-context.

Tijdsplanning:

- Fase 1: 3 weken - requirements-analyse en dataverzameling.
- Fase 2: 2 weken - architectuurontwerp en selectie van tools.
- Fase 3: 6 weken - implementatie van proof-of-concept.
- Fase 4: 2 weken - evaluatie, metingen en rapportering.

Deliverables per fase:

- Fase 1: Analyseverslag van huidige situatie en probleemstelling.
- Fase 2: Ontwerpspecificatie van de architectuur en datastromen.
- Fase 3: Werkend proof-of-concept systeem.
- Fase 4: Evaluatierapport met KPI's, conclusies en aanbevelingen.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat is een proof-of-concept van een gecentraliseerd securitymonitoringsysteem dat:

- Securitydata van het geselecteerde SaaS-platform en de DevOps-pipeline centraliseert en normaliseert;
- Incidenten sneller en betrouwbaarder detecteert via correlatie en rule-based analyse;
- Automatische rapportages genereert voor technische teams en management;
- KPI's levert zoals MTDD, MTTR, aantal false positives en responstijd.

Eenvoudige anomaly-detectie wordt optioneel onderzocht, als aanvulling op rule-based detectieregels. Uitgebreide ML-methoden vallen buiten de scope van deze bachelorproef en worden enkel kwalitatief besproken indien toegepast.

De meerwaarde voor de doelgroep ligt in hogere operationele efficiëntie, betere risico-inschatting en snellere incidentafhandeling. Het onderzoek levert praktische inzichten en een concrete demonstratie van een geautomatiseerd monitoring- en incident response-systeem dat haalbaar is binnen de beschikbare tijd en middelen, dankzij de beperkte en realistische scope.

Bibliografie

- Billawa, P., Bambhore Tukaram, A., Díaz Ferreyra, N. E., Steghöfer, J.-P., Scandariato, R., & Simhandl, G. (2022). SoK: Security of Microservice Applications: A Practitioners' Perspective on Challenges and Best Practices. *Proceedings of the 17th International Conference on Availability, Reliability and Security*. <https://doi.org/10.1145/3538969.3538986>
- Chuvakin, A. (2010). *The Complete Guide to Log and Event Management* (White Paper) (Sponsored by NetIQ). NetIQ. https://www.netiq.com/en-gb/docrep/documents/m47h82fbmy/the_complete_guide_to_log_and_event_management_wp_ee.pdf
- Denning, D. E. (1987). An Intrusion-Detection Model. *IEEE Transactions on Software Engineering*, SE-13(2).
- Fuentes-García, M., Camacho, J., & Maciá-Fernández, G. (2021). Present and Future of Network Security Monitoring. *IEEE Access*, 9, 112744–112760. <https://doi.org/10.1109/ACCESS.2021.3067106>
- García-Teodoro, P., Díaz-Verdejo, J., Maciá-Fernández, G., & Vázquez, E. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers Security*, 28(1). <https://doi.org/10.1016/j.cose.2008.08.003>
- González-Granadillo, G., González-Zarzosa, S., & Diaz, R. (2021). Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures. *Sensors*, 21(14). <https://www.mdpi.com/1424-8220/21/14/4759>
- Jakku, P. C. (2025). Transforming DevOps: The Critical Role of Monitoring and Logging in Effective Troubleshooting and Optimization. *International Journal of Global Innovations and Solutions (IJGIS)*. <https://doi.org/10.21428/e90189c8.dc6056f7>
- Kent, K., & Souppaya, M. (2006). *Guide to Computer Security Log Management* (NIST Special Publication Nr. 800-92). National Institute of Standards en Technology. Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-92>
- Kral, P. (2012, februari). *Incident Handler's Handbook* (White Paper). SANS Institute. <https://www.sans.org/white-papers/33901>
- National Institute of Standards and Technology. (2018). *Framework for Improving Critical Infrastructure Cybersecurity* (tech. rap. Nr. Version 1.1). NIST. Gaithersburg, MD.

- National Institute of Standards and Technology (NIST). (2025). NIST SP 800-61r3 Incident Response Recommendations and Considerations for Cyber Risk Management [Incident response lifecycle based on CSF 2.0 Functions]. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r3.pdf>
- Nelson, A., Rekhi, S., Scarfone, K., & Souppaya, M. (2025). *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile* (Special Publication Nr. 800-61r3). National Institute of Standards en Technology. Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-61r3>
- Obregon, L. (2015). *Infrastructure Security Architecture for Effective Security Monitoring* (White Paper). SANS Institute. <https://www.sans.org/white-papers/36512>
- Persistence Market Research. (2025). IT Infrastructure Monitoring Market Size, Share, and Growth Forecast, 2025–2032 [Accessed 2026].
- Prates, L., & Pereira, R. (2025). DevSecOps practices and tools. *International Journal of Information Security*, 24(1), 11. <https://doi.org/10.1007/s10207-024-00914-z>
- Raponi, S., Caprolu, M., & Di Pietro, R. (2019). Intrusion Detection at the Network Edge: Solutions, Limitations, and Future Directions. In T. Zhang, J. Wei & L.-J. Zhang (Red.), *Edge Computing – EDGE 2019*. Springer International Publishing.
- Scarfone, K. A., & Mell, P. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)* (tech. rap. Nr. NIST SP 800-94). National Institute of Standards en Technology. <https://doi.org/10.6028/NIST.SP.800-94>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities* (NIST Special Publication Nr. 800-218). National Institute of Standards en Technology. Gaithersburg, MD, USA. <https://doi.org/10.6028/NIST.SP.800-218>
- Staniford-Chen, S., & Heberlein, L. T. (1998). Holding Intruders Accountable: Active Response in Computer Security. *Proceedings of the 1998 IEEE Symposium on Security and Privacy*.
- Tariq, S., Baruwat Chhetri, M., Nepal, S., & Paris, C. (2025). Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities. *ACM Comput. Surv.*, 57(9). <https://doi.org/10.1145/3723158>
- Tuyishime, E., Balan, T. C., Cotfas, P. A., Cotfas, D. T., & Rekeraho, A. (2023). Enhancing Cloud Security—Proactive Threat Monitoring and Detection Using a SIEM-Based Approach. *Applied Sciences*, 13(22). <https://www.mdpi.com/2076-3417/13/22/12359>
- Vardalachakis, M., Vasilakis, M., & Tampouratzis, M. (2025). A Comprehensive Analysis of Features, Benefits, Challenges, and Best Practices of Security Information

and Event Management (SIEM) Solutions. *Computer Sciences amp; Mathematics Forum*, 12(1). <https://www.mdpi.com/2813-0324/12/1/18>